

SemIO

A domain-specific programming language for
architects

by

Ueli Saluz

A thesis presented for the degree of
Bachelor of Science Architecture

to

Institute of Architecture
Faculty VI Planning and Building Environment
Technische Universität Berlin

Supervised by:

Prof. Raoul Bunschoten

Co-supervised by:

Prof. Dr.-Ing. Philipp Geyer

CHORA
conscious city



Abstract

The construction industry today is facing great challenges. The construction projects are complex, the value chains global and the ecological responsibility of the construction sector is great. These challenges can only be mastered if planning is more efficient, transparent, and sustainable. The role of the architect has a great deal of responsibility, as his task is overall planning. He defines the interfaces and manages the planning and construction process. The success of a construction project depends largely on how the interfaces are designed. Digital tools have proven to be extremely powerful for effective and fault free exchange of data. An automated data flow can only be achieved if all the actors involved can make their expertise digitally available to each other.

The design logic behind concrete design decisions is usually referred to as an architectural concept. Two main tools of the architect are the plan and the model. They are the condensate of much information that has played a role in the design process. Over the last decades, many software solutions have been developed to capture, modify, and exchange plans and models. Some of these solutions make it possible to digitally define not only the resulting model, but also the architectural concept behind it. This, however, has one major limitation: the design logic must be exclusively formal in nature. To be able to digitally represent architectural concepts that work with non-formal design logic, new notation methods are required.

This thesis presents a conceptual notation method that allows to digitally define project-specific abstraction patterns. SemIO is a programming language that not only represents a design tool, but also defines the interfaces in the planning process. It thus closes the current gap of the architect in the digital planning and construction process and allows digital continuity already in early planning phases through automated data exchange. Due to clearly defined in- and outputs, design tasks can be parallelized which leads to a dramatic reduction of the planning phase. A prototype was developed that understands SemIO programming code and automatically creates a digital Document Object Model (DOM). This DOM is directly used to generate a program in other object-oriented programming languages.

Zusammenfassung

Das Bauwesen steht heute vor großen Herausforderungen. Die Bauaufgaben sind komplex, die Wertschöpfungsketten global und die ökologische Verantwortung der Baubranche ist groß. Diese Herausforderungen können nur bewältigt werden, wenn effizienter, transparenter und nachhaltiger geplant wird. Der Rolle des Architekten kommt dabei viel Verantwortung zu, da seine Aufgabe die Gesamtplanung ist. Er definiert die Schnittstellen und leitet den Planungs- und Bauprozess. Der Erfolg eines Bauprojekts hängt maßgeblich davon ab, wie die Schnittstellen gestaltet sind. Um effektiv und fehlerfrei Daten auszutauschen, haben sich digitale Werkzeuge als äußerst mächtig erwiesen. Ein automatisierter Datenfluss kann nur erreicht werden, wenn alle beteiligten Akteure sich gegenseitig ihre Kompetenz digital zur Verfügung stellen können.

Die Entwurfslogik hinter konkreten Entwurfsentscheidung wird meist als architektonisches Konzept bezeichnet. Zwei Hauptwerkzeuge des Architekten sind der Plan und das Modell. Sie sind das Kondensat vieler Informationen, die im Entwurfsprozess eine Rolle gespielt haben. In den letzten Jahrzehnten wurden viele Softwarelösungen zum Festhalten, Verändern und Austauschen von Plänen und Modellen entwickelt. Manche dieser Lösungen erlauben es, nicht nur das resultierende Modell, sondern auch das dahinter liegende architektonische Konzept digital zu definieren. Dies jedoch mit einer großen Einschränkung: Die Entwurfslogik darf ausschließlich formaler Natur sein. Um architektonische Konzepte, welche mit nichtformaler Entwurfslogik arbeiten, digital abbilden zu können, braucht es neuartige Notationsmethoden.

Diese Arbeit stellt eine konzeptuelle Notationsmethode vor, welche es erlaubt, projektspezifische Abstraktionsmuster digital zu definieren. SemIO ist eine Programmiersprache, welche nicht nur ein Entwurfswerkzeug darstellt, sondern gleichzeitig auch die Schnittstellen im Planungsprozess definiert. Sie schließt damit die gegenwärtige Lücke des Architekten im digitalen Planungs- und Bauprozess und erlaubt bereits in frühen Planungsphasen digitale Kontinuität durch automatisierten Datenaustausch. Durch die klar definierten In- und Outputs können Entwurfsaufgaben parallel bearbeitet werden, welches zu einer erheblich verkürzten Planungsphase führt. Es wurde ein Prototyp entwickelt, der SemIO Programmiercode versteht und daraus automatisch ein digitales Dokument Objekt Modell (DOM) erstellt. Dieses DOM wird direkt zur Generierung eines Programms in einer anderen objekt-orientierten Programmiersprache verwendet.

Contents

1	Introduction	1
2	Planing and Modelling	4
2.1	Plans and Computer Aided Design	7
2.2	Models and Building Information Modelling	9
3	Design	11
3.1	Paradigms	12
3.1.1	High-level	12
3.1.2	Domain-specific	13
3.1.3	Object-oriented and functional	14
3.1.4	Compiled	14
3.1.5	Open-source	15
3.2	Concepts	16
3.2.1	Project	18
3.2.2	Abstraction levels	18
3.2.3	Things	21
3.2.4	Schemes	22
3.2.5	Parameters	23
3.2.6	Indices	25
3.2.7	Benchmarks	25
3.2.8	Functions	26
3.2.9	Conditions	26
4	Prototype	28
4.1	Syntax	28
4.1.1	Line indentation	30
4.1.2	Dot notation	30
4.1.3	Bracing	30
4.2	Implementation	31
4.2.1	Regular expressions	32
4.2.2	Document object model	33
4.2.3	Dynamic linked library	33
4.2.4	Design patterns	33
4.3	Example	33
5	Conclusion	37
A	Appendix	I
A.1	Statistics	I
A.2	Compiled code	IV

Computer science glossary

programming language

A programming language is a formal language that can be understood by a computer and be executed at a point. [11](#)

formal language

A formal language is a mathematically defined language that follows clear rules and doesn't leave room for interpretation. [11](#), [14](#)

code

Programming code, also called programs, describes a simple text document that is written in a certain programming language. [14](#), [15](#), [31](#)

implementation

An implementation is code that can deal with a certain abstract problem. [11](#)

compiler

A compiler translates code from one (more human) programming language completely into code from another (more machine understandable) programming language.. [2](#), [15](#), [32](#)

parser

A parser chops a text file into logical pieces of information. With this pieces the compiler can start to relate the pieces between each other.. [32](#)

pattern

A pattern is a way of solving a certain problem that lays in specific context. [3](#)

model

A model is an abstraction of objects.. [9](#), [10](#), [14](#), [22](#)

Application Programming Interface

An Application Programming Interface allows a programmer to extend functionality of this software application. This can simply add new features or be a new interaction with another software. [7](#)

paradigm

A paradigm describes a way of programming. If you can achieve this way of programming with a programming language then you can say: This language is [name of paradigm]. [14](#)

object oriented

Object orientation is a paradigm. Object oriented programming languages support to virtually create models with certain properties and abilities. [vii](#), [9](#), [14](#), [32](#), [33](#)

functional

Functional is a paradigm. Functional programming languages support mathematical function composition. (You can give your function for x another function as a variable instead of a value and your function can return a function instead of a value). [14](#)

framework

A framework is a library o provides the basis for software. contains generic functionality that the specific software can use. [9](#), [11](#), [19](#), [24](#), [31](#), [32](#)

machine code

The machine code is the last code that tells the computer (CPU) what to do. Machine code contains instructions about where to really store values and do compute simple arithmetic operation. [12](#)

expressivity

Expressivity describes how much code do you need to describe a problem and how understadable this code is if you don't know the programming language. (I am not refering to theoretical expressivity) . [13](#)

Architectural glossary

project

An architectural project is formalised through architectural designs. 5, 8, 9, 16

design

An architectural design is a formalised architectural idea through an architectural concept. 5

architectural concept

An architectural concept describes the logic behind an architectural design. 3, 5, 7, 10, 11, 19

form

Form is the concrete appearance of an object. 5, 8, 9, 14

plan

An architectural plan is a planar projection of virtual architectural objects. 7, 13, 15

model

An architectural model is a formal representation of an architectural object. 9, 14

Building Information Modelling

Building Information Modelling means digitally sharing all digital data around a building. This ranges from the first draft until the demolition of the building. vii, 8

International Foundation Classes

The International Foundation Classes are the main international standard for data exchange of digital models. It defines how architectural objects are abstracted. vii, 9

Acronyms

CAD	Computer Aided Design.	6, 8, 12
BIM	Building Information Modelling.	8, 10, 14, 20
IFC	International Foundation Classes.	9, 11, 24
LOD	Level of Detail.	20
AEC	Architecture Engineering Construction.	8, 14
CGA	Computer Generated Architecture.	14
GUI	Graphical User Interface.	11, 32
DSL	Domain Specific Language.	13, 19
GPL	General-Purpose Language.	13
OO	Object oriented.	14
DLL	Dynamic Linked Library.	32
DOM	Document Object Model.	24, 32, 33

1. Introduction

In the Renaissance, as now, new information technologies, instead of building technologies, were the agents of change. New information technologies brought about some new devices of design that in their turn revolutionized the process of building and changed architectural forms.¹

In the following I am proposing a new programming language *SemIO* as a new device of architectural design. But before I start, I want to highlight that the entire thesis is written to be understandable for non-computer scientists and non-architects. SemIO only makes sense if architects understand what I am introducing with the tools of computer science and vice versa. Therefore, in case there are terms you don't know or you understand differently, check out my glossaries to see what I am referring to. Sometimes the same term has a different definition depending if it is used in the context of architecture or computer science. In case you don't find the definition inside the chapter or inside the glossary the term will not be further relevant. You will probably notice that all these definitions are kept abstract. My intention is not that you technically understand what they are. For that you would have to use them in practice. But instead I want you to be able to relate these definitions between one another. Once you spin all of these definitions together, the resulting web will be what SemIO really is about. It is important to read this thesis not as analytical work but as a *project*. It is not an *architectural design* but a *design for architecture*. The first chapter is referring to the epistemological base of SemIO and the aim is to better understand the knowledge behind SemIO. Furthermore I did not invent definitions of the concepts that I will be about to explain. The definitions are filtered and modified for what I believe is necessary to understand SemIO.

Let me begin with explaining the relationship between a computer and programming languages. When we speak of computers we usually refer to machines that are able to compute something for us. This computation physically happens on a certain hardware but this would be worthless if the computer didn't have a programmable interface with software. Here is what you need to know: Computers are still stupid. Their intellectual capacity basically doesn't exist but their ability to do

¹This is part of a transcript of a presentation that Mario Carpo, a contemporary architectural historian and critic, gave at the Canadian Center for Architecture in 2004 ([Goodhouse & Canadian Centre for Architecture 2017](#), p.35-36)

simple arithmetic operations is overwhelming.² In order to demonstrate the evolution of computational power I give a little example. The Z3, which is considered as one of the first computers and was actually developed by the building engineer Konrad Zuse because he wanted to automate his complicated statical computations³, could add three times two already quite large numbers⁴ in around two seconds.⁵ 80 years later a cheap computer can easily add 400 000 000 000 times two even larger numbers in one second.⁶ How can we make use of that great computational power? We need to find a way to translate our real world problems and turn them into a list of simple arithmetic operations. And this is what programming languages do. They accept a certain format of text and translate this text into a list of commands so that the computer can execute them. In reality there is not only one programming language but many chained after each other. Just the last programming language really tells the machine what to do.

Maybe this gets more clear with a simple example. Assume you are the captain of a big sailing ship with a large crew. It is your job to get the ship sailing and set the course. In order to start the journey there are plenty of actions that need to happen like weighing the anchor, hoisting the sails, etc. Each of these actions needs to be adjusted to the specific vessel and specific context like wind power and water drift, etc. You don't want to give everyone their personal instruction and even if you wanted, you wouldn't have the time to. So instead you use certain keywords that represent procedures. Every time you shout a certain keyword other people decide according their context, what other instructions to give further until nobody says anything any more. The job is done. In theory you don't even have to know how to sail as long you know the keywords.

Now imagine that the ship is sailing means, the computer is calculating. The language you speak is a programming language. The mate who translates your commands into subcommands is a [compiler](#)⁷. These subcommands are in a different programming language that you don't need to know. Your computer contains mainly of the poor guys in the end who actually pull the ropes. You might guess that you have a huge amount of idle sailors in your computer. So your job is to efficiently set interfaces and hierarchy levels to keep all sailors busy. The closer the language is to the captain, it is referred as a high level programming language. The closer it is to the sailor, it is referred as a low level programming language.

You might guess that the higher you go, the more specific it gets, the less freedom you have. It is obvious because the less information you give, the more you rely on the others doing their job properly. But properly is not an objective term. Sometimes

²The power of AI, in this case I use it as synonym to machine learning, also comes from simple arithmetic operations. One mathematical foundation is to find an appropriate mathematical function with multiple variables, multivariate regression, that describes a specific problem based on concrete/discrete cases to a continuous abstract mathematical model.

³According to following article ([Kurrer 2001](#), p. 577) Zuse wanted to automate statically indeterminate computations.

⁴Already the Z3 could store floating point numbers from the range of $1.08 \cdot 10^{-19}$ to $9.2 \cdot 10^{18}$ ([Rojas 1996](#), p 305). In comparison the normal floating point value in a c-language has the range from $3.4 \cdot 10^{-38}$ to $3.4 \cdot 10^{38}$ [Microsoft \(2020b\)](#)

⁵According to ([Rojas 1996](#), p 306) the clock speed of Zuse Z3 was around 5Hz and the addition took 3 clock cycles.

⁶I assumed a computer with following CPU: AMD Athlon™ Silver 3050U [Advanced Micro Devices \(2020\)](#) which has according to [Boehme GbR \(2020\)](#) a FP32 Index of 420. The FP32 Index is measured in GFLOPS (giga float operations per second). One addition counts as a float operation.

⁷In this example it also could be an *interpreter* but as I will show later my compiler prototype, I will not explain the difference between those two here.

one solution might work for now. Depending on the way you solved a problem, you might get into troubles if you keep on developing it.

Architects know this problem very well. That's why it is important to design with a solid [architectural concept](#). In my studies this was probably the most important criteria for all the designs I had to do. In computer science an architectural concept is called software design pattern, short [pattern](#). They were mainly introduced by the so called Gang of Four.⁸ Actually they refer their definition of *design pattern* explicitly to the definition of the architect Christopher Alexander:

Christopher Alexander says, "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice"⁹

Note the important last part of the sentence "without ever doing the same way twice". This reveals one very important property of a design pattern that might be not intuitive. The goal is not to copy and paste a solution but the opposite. Patterns are there to prevent repetition but at the same time give an abstract tool to learn from the past.¹⁰ They formulate a *way behind* solving a problem, not just how to solve it. Later you will see that this property is a very important one for SemIO.

⁸In the foreword of the book Design Patterns: Elements of Reusable Object-Oriented Software (Gamma et al. 1994, p 9) the surname Gang of Four appears for the first time.

⁹(Gamma et al. 1994, p 12) Alexander used the term pattern already in his early book: Notes on Synthesis of Form Alexander (1964) but the Gang of Four refers to the definition of a pattern in (Alexander et al. 1977, p. x)

¹⁰For me the great strength of Christopher Alexander's pattern theory is that he proposes an abstract textbased notation method in order to describe an architectural concept. The biggest mistake he does according to me is to believe that there are cultural independent *good* patterns but his theory is much stronger. In order to explain that, I would have to go much further and this would go out of the scope of this thesis.

2. Planing and Modelling

Now here is the challenge: In its long history, design practice has done a marvellous job of inventing the practical skills for drawing objects, from architectural drawing, mechanic blueprints, scale models, prototyping etc. [...] , and yet, four hundred years after the invention of perspective drawing, three hundred years after projective geometry, fifty years after the development of CAD computer screens, we are still utterly unable to draw together, to simulate, to materialize, to approximate, to fully model to scale, what a thing in all of its complexity, is.¹

What Bruno Latour describes here accurately is the original problem that this thesis is trying to solve. While his concern was about how to conceptually model *things*², my concern is how to conceptually model *architectural things*. Philosophers and computer scientists have a lot of problems and methods in common. One being the question how do we abstract the world in a logical way and which concepts do we need in order to describe this abstracted world. While both are trying to find appropriate models to explain the world, their intentions are very different. The philosopher aims to gain theoretical knowledge. The computer scientist on the other hand has very practical intentions. Without an abstract model of reality there is no way that the computer can interact with reality.³ In order to create such abstract model we need to find appropriate concepts.

¹(Latour 2012, p.162)

²He refers to the definition of Heidegger and his main intention is that "we know how to draw Gegenstand but we have no clue what it is to draw Ding [thing]." (Latour 2012, p. 163)

³Actually Latour accurately marks out that CAD only tackles objects in the heideggerian sense of (Gegenstände). "We know how to draw, to simulate, to materialise, to zoom in and out on objects; we know how to make them move in 3-D space, to have them sail through the computerized virtual *res extensa*" (Latour 2012, p.162). While he says that: "I am not advocating for another CAD design for Prometheus." (Latour 2012, p.163) My intension with SemIO is exactly to extend our *res cogitans* digitally by introducing a digital notation tool for architectural *things* (*Dinge*) and not only *Gegenstände*. When Latour says that: "computerized virtual [...] space [...] is the most utopian (or rather atopic) of spaces. These are the least realistic spaces of circulation ever imagined. They are spaces that do not even fit with the ways in which architects, engineers, designers draw and modify blueprints, nor with the process through which they direct fabrication on the factory floor or manipulate scale models." (Latour 2012, p.162-163). I don't believe this needs to stay like this.

Architects design *projects*.^a A *project* is formed by *designs*. In order to express *design* you need *notations* to represent it, otherwise they remain ideas. Ideas are translated through an *architectural concept* into *design*.^b A *project* is therefore always indirectly represented by *notations*. The *notations* contain documented *information* about the *project*. In order to create *notations* of *designs* we need *information technologies*. An *information technology* is a *technology* for systematically dealing with *information*.^c

^aThe epistemology of Bojanić (2019) about the concept *project* heavily influenced all my definitions.

^bThe concept *architectural concept* is in my experience very common under architects. Vesnić (2017) gives more insights about what an architectural concept is and investigates the properties further in Vesnić & Ćipranić (2019).

^cI am mean the literal sense of information technology and not as a synonym for digital IT.

To explain that lets take a look of an etymological meaning of *project* and *design*.⁴ The Latin origin from *project* is prefix *pro-* which means in this context *forward* and *ject* comes from *iacere* which is the past participle from *iactus* and means throwing. A *project* is something that gets thrown foreword. This means there has to be a standing point and a direction in which a project is thrown, projected. The standing point is given by time: the present. The direction is the future. If the direction would be the past, it would be historical. That's why a project is always progressive and can't be *perfect* or to give a synonym *past*. An *architectural project* is therefore a projection of architectural designs into the future.

What about the concept *design*? The Latin prefix *de-* can indicate that a *sign* has been put down. This gives us both important aspects of the concept design. First it needs to be a *sign* and second that this *sign* has found *form*. Therefore design is referential to an idea and concrete through form. This clearly separates idea from design. An idea doesn't need to be formalized in comparison to a design.

Projecting can only happen through the use of *information technologies*. For architectural design such information technologies can be for example plans, models, images, diagrams or text. In this chapter I will focus on plans and models. We will take a look at current digital implementations and their relation to analogue implementations.⁵

The main intention of this chapter is to show that tools, in this case notation methods, are not just tools. The potentiality of a design lies in the potentiality of the

⁴Etymology is for me a construction that needs to be done a posteriori. That's why I call it "an" etymological meaning, that I based on appearance in history. I took these appearances for *project* from the same German word *Projekt* in Pfeifer et al. (2020c) and in English from Harper (2020e) and for *design* as well the German word *Design* Pfeifer et al. (2020a) and in English Harper (2020c). The author Douglas Harper is not formally educated and says: " I am no etymologist. [...] When you do something vast and complex with no formal training, you know you're going to be wrong a lot. " Harper (2020b) For my use epistemological purpose this dictionary is excellent.

⁵With analogue I mean the literal opposition of digital. Analogue describes a technique where there controlling involves no discrete digital numbers. I am not referring to Goodman (1976) definition with: "Plainly, a digital system has nothing special to do with digits, or an analog system with analogy." I mean exactly this plain definition.

tool being used to design.⁶ What do I mean with potentiality? You can think of every possible and impossible design but you can only test it if there is a tool to test it. It doesn't matter if you are aware of the limits of your tool or not. The potentiality is already defined before you start designing with the tools you are using. Note that potentiality has nothing to do with the resulting size or appearance. Potentiality is about how informations can be connected. Not about possible *phenotypes* but about possible *genotypes*.

It is important to know that when I speak of plans and models, I mean the information technology itself and not the analogue or digital way of using such. For example [Computer Aided Design \(CAD\)](#) is a tool for the information technology plans. The potentiality of analogue planning is just a fraction of the potentiality of digital planning. Both of these potentialities are bound by the overall potentiality of the information technology plans.

In an analogue practice there is no explicit linking between the notations. Later in this chapter we will see that digitally linking notations completely changes the properties of those. As long as notations are not linked and individually refer to the project, there is no guarantee that the merged result is perfectly congruent. On one side this offers freedom during the design process because this allows errors and only demands precision in the construction phase when information for physically building is needed. On the other side this can cause great problems in later planing phases when substantial errors were not noticed. The core question is: How much precision do I need in order to make *adequate* decisions?

An important property of plans and models which deals with precision is scale. Scale naturally give us a way to think in abstraction levels. If you wish to add details of a window frame which has a thickness of 10 cm in a 1 to 500 scaled plan or model this will have the width of 0.2 mm. Neither your pen can draw nor your chisel can carve that. It is a pointless information in this scale. This natural barrier disappears with digital plans and models. When you can zoom in and out, you can add any detail at any point. This forces us to rethink if scale is a necessary property of the plan or model or not.⁷

To answer the question what an *adequate* decision is, let's again take a look at the etymology. The prefix *ad-* can be understood as *fits to*, *equate* is related to Latin *aequare* and means something like make equal/make level/make even.⁸ In this context, an *adequate* decision fits to an abstraction *level*. That makes *adequate* precision a relative term.⁹

Abstraction levels inside plans and models have great impact on the way architects design. Not only do they control the way decisions are taken but probably even more

⁶[Vesnić \(2017\)](#) uses the concept *potentiality* in regards to an *architectural concept* which she derives from [Deleuze & Guattari \(1994\)](#): “The philosopher is the concept's friend; he is potentiality of the concept.” In my case the philosopher is the information technology and the concepts are design tools.

⁷My answer to this question is closely related to the design of SemIO and I will go more into detail in the section [Abstraction levels](#)

⁸[Harper \(2020a\)](#) See my other footnote about etymology [4](#)

⁹I will not go into detail about the long debate what formal consequences this has. [Garcia \(2014\)](#) categorizes this debate following: “Architectural styles and theories have cycled and swung repeatedly between those supporting less or no detail (and generally more rather than less reduction, abstraction, suppression, universality, continuity, unity, clarity, simplicity, coherence, purity, honesty, terseness and homogeneity), and those supporting more detail (and generally more rather than less expression, articulation, eclecticism, ad hoc, variety, complexity, disjunction, specificity and heterogeneity).”

important they help to interact with other architects. Most of the time designing a building is not a solitary task. In this case architects need to eventually communicate with other architects. For effective communication you need to know what you are talking about. When designing with scale the boundary of possible decisions is implicitly set. When designing without scale you always have to explicitly frame what your goals are. Otherwise you run the risk designing different things.

But before I start explaining plan and model more in detail, I want to say something general about software. This impacts all digital interfaces. Software is build for a purpose that implies a certain type of user and a certain use case. The wider the type of user or the use case, the less specific the support for certain tasks can be, or the more explicit informations must be given. Freedom is not something that is per se good. It always comes with the effect that you need more technical knowledge and more time to develop it. When I say that a software doesn't support a certain task or doesn't contain a feature, this means only there is no native, direct way of solving this problem without being a programmer. Most software contains an [Application Programming Interface](#) with which you can extend the program for your functionality. But for that you have to know about programming. Therefore when a software doesn't offer something it is not about the question whether it is possible or not, but only if it is implemented or not.

2.1 Plans and Computer Aided Design

Let me begin with the probably most important abstraction tool of architecture in the past 500 years: The [plan](#). Often the plan is considered as the language of architects and engineers. It is important that this should be understood literally and not seen as a metaphor. To learn a language you need to learn symbols, the corresponding syntax and the semantic. Symbols, mainly lines and fillings, have different meanings according their context. Sometimes they represent walls, sometimes human beings, sometimes trees and sometimes they don't allow a clear interpretation. Not only the contour is relevant but the way how lines and fillings are drawn contains a lot of meta-information. The line width is one example. One interpretation of the line width could be that a wall is bearing or non-bearing, massive or dry-built. In case there is only one type of wall, the architect can decide to use the line widths to encode different information. Which of the identical walls has higher importance than others? What conceptual space do the walls belong to? This ambiguity means great freedom for the architect. He can become a poet and overlap meanings, create neologisms and find a creative way to partly formalize an [architectural concept](#). The amount of information inside a plan is in theory unlimited because the set of symbols can be extended.

In reality the limit is reached when you have so many lines that your paper or your screen is black.¹⁰ When you store too much information nobody can read your plan any more. This makes the plan obsolete because the main function of the plan is communication. Something crucial for plans is that they are planar. They only *represent* a fraction of three dimensional data on a two dimensional plane. This brings a problem along. An individual plan can never capture a full project. Therefore it needs to be linked to other plans. This task usually happens in the head of the one who reads the plans. Part of architectural education is about learning how to

¹⁰With scaling you could solve this problem. I will talk about later.

extract a project from multiple plans. This has the effect that plans are an exclusive way of communication. Earlier we saw that the notation rules for designing are not standardized. In reality this encoding is an encryption. It needs further knowledge about the way that information is encoded, the key, to decode the plan again. Besides the knowledge about general notation there is something more important: You need experience. First to be able to guess the key for individual notation rules and second to actually be able to image the [project](#) behind it. This requires great spacial imagination power. You might learn pretty fast how to read musical notation but to hear the music behind it, is a complete different story.

After looking at entities of the plan let me relate to digital interfaces. Before computers existed, plans were hand drafted and the easiest way to turn them digital would be to scan them and create a pixel based representation but there is no specific software that deals with pixel based architectural plans. For good reason. Plans are an encoded way of storing architectural information and scanning adds another encoding layer. Encoding the encoded informations would make the access and modifying even harder. We saw that plans only form a project in combination with other plans. In order to build a project, it requires to reassemble them. If you want to digitally use plans as a generation tool of [form](#) and not just as a representation tool, you have to find a way to link all the partial informations into one database.

There are two ways this problem is solved in software today. First a *dynamic* and second an *undynamic* linking between plans and model.

At this point I need to introduce the concept [CAD](#). The term itself is very unspecific. I don't want to go into the history of the term and instead focus on the meaning of today in the field of architecture. Autodesk, one of biggest software firms in [Architecture Engineering Construction \(AEC\)](#) advertise their AutoCAD with: " AutoCAD is computer-aided design (CAD) software that architects, engineers, and construction professionals rely on to create precise 2D and 3D drawings ".¹¹ This definition fits to my experience. Although the term Computer Aided Design could incorporate a way larger meaning, it is commonly reduced to software that deals with digital 2D and 3D drawings.¹²

Undynamic linking would be any CAD software that supports 2D and 3D geometric objects and offers the functionality to turn a 2D object into a 3D object. Once the operation is finished, a new independent 3D object has been created. If you modify the original 2D object it will just affect this specific 2D object and no objects derived from it. Nowadays all commonly used CAD software supports that. Software with dynamic linking keeps tracks of established links between an object and its derived objects.

Dynamic linking completely changes the entities of plans. Undynamic linked or unlinked they are a free, individually definable and exclusive design tool of first order and suddenly they turn into a standardized representational design tool of second

¹¹That is the description from the website. [Autodesk \(2020a\)](#) I took AutoCAD as a representation for CAD as Autodesk had the long history in the field of CAD software. They have developed the Drawing Interchange File Format (DXF) [Autodesk \(2020b\)](#) standard for data exchange in CAD software. In 2016 Autodesk had the highest CAD market revenue share worldwide [A.1](#). This revenue is not only based on AutoCAD as their CAD software portfolio is bigger but AutoCAD is for architecture their main CAD product. Note that Revit doesn't count as CAD software but as [Building Information Modelling \(BIM\)](#) software.

¹²For example ArchiCAD still contains the CAD in the name but Graphisoft advertises ArchiCAD as "the most efficient and intuitive BIM software on the market" [Graphisoft \(2020\)](#). In Japan ArchiCAD was according to [A.2](#) with around 40% the most popular software in 2019.

order. Second order because unlinked the plans would generate the model but linked it is the other way around. The model is not the representation of the plans any more but the plans are the representation of the model. This has quite some consequences. The potentiality of designing is no longer bound by the potentiality of plans but it is bound by potentiality of models. You can no longer build what you can draw but instead you can build what you can model. In other words the architect no longer *plans* but started to *model*.

2.2 Models and Building Information Modelling

We saw what happened with a plan when it gets linked to a model. Now we will take a look at what a *model* really is. What *modality* has a model? While an *architectural model* is generally something geometrical, a *model* in computer science has no geometrical *form*. It is purely abstract. Depending on the domain there are many more definitions of *model*. What do all of these definitions share?

If we take a look at the etymology, then we see that *model* is related to Latin *modullus* which is the diminutive from *modus*.¹³ *Modus* can be understood as manner or measure and *modullus* is therefore a small standardized measure. This gives us the important properties that models are small measures of something, and are representative for a manner. Thus they are a simplified (small standardized) concrete (measure) representation of something general (manner).

In order to represent something, there is something that needs to be represented. Whether the abstracted objects represent the concrete objects or the other way around is a philosophical question but anyhow the model forces you to think in abstract objects. In architecture abstract objects are formulated through geometry. In computer science abstract objects are formulated through text (programming code) or diagrams.¹⁴ The *modus* to think in *models* is referred as *object oriented*. It is not surprising that when architectural modelling became digital, the underlying *framework* would be created based on models of building elements.¹⁵

What are the properties of an architectural model? In the section before we saw that plans are an *exclusive* way of communication due to their free and unstandardised notation rules. Experience is needed to be able to imagine the *project* that they refer to. How about the model? First it seems like this barrier goes away because 3D data is no longer being projected but represented as 3D data again. This takes one layer of complexity off. Yet the most complex part stays: Imagining the *project* behind it. This requires the same experience as before.

While a plan is selective because it only shows a fraction of informations about the *project*, the model can in theory contain all information about a project. In an analogue practice there is only one physical model that contains all the information of a project. That is the built project itself.¹⁶ In every other physical model abstractions

¹³In english: Harper (2020d), In german: Pfeifer et al. (2020b) See my general comment on etymology. ⁴

¹⁴There is a wide known separate graphical programming language, UML, which is developed from the Object Management Group (2020) that standardizes notation and allows to turn code into diagrams and the other way around.

¹⁵I am referring to the main framework of BIM: International Foundation Classes (IFC) which was developed from buildingSMART International, Ltd (2020b). There are few examples like the IfcZone class buildingSMART International, Ltd (2020a) but most of the classes are based on building elements

¹⁶This is not really an architectural *model* but i'll cover that in a little bit.

have to be made in materiality, construction and scale. How scale, better: abstraction levels, impacts the precision during the design process and why relative precision is important for designing was covered earlier. [2](#)

In a digital practice it suddenly becomes possible to actually build a full model with all material information and geometric information without having to really build it. This is what [BIM](#) is. Before talking about the consequences this has for the design process, I want to ask the question what kind of *model* such a model with all information is? In fact a model with no abstraction is no model. To be more precise: This model is still a [model](#) in computer science because the abstraction happens from real to digital, but it is not an *architectural model* any more.

Therefore if the architect wants to keep on thinking abstract and expressing *design* through the information technology *plan* and *model*, the only way to keep the freedom is to design analogue and then once done with designing, put all the information into a digital container. This would be exactly what BIM is trying to prevent because every time the same information gets manually copied, mistakes happen. Further this "digital" way of planing helps with building but when it comes to use the computational power for design purposes, no improvements have been made because the *designing* didn't happen digital. To allow BIM for architects new tools must be found in order to translated [architectural concepts](#) into a digital format.

My proposition with SemIO is to find a new to way to encode the information that are behind designing plans and models into a different information technology. This information technology is text. Text is abstract and has the potentiality to fill the gap of abstraction that digital "models" are missing in the moment. The intention of SemIO is to turn BIM into conceptual BIM that can be used by the architect as a *design* tool.

3. Design

In this chapter I will explain the design behind SemIO. This contains the core concepts and the supported paradigms that this [programming language](#) is supposed to offer. Together they form the [architectural concept](#) of SemIO. This part is purely abstract and has nothing to do with the actual [implemenation](#). How the architectural concept turned into code will be covered in the fourth chapter where the [Prototype](#) will be explained.

In the introduction I explained the general relationship between a computer and programming languages. [1](#) The term programming language is an open term. Any kind of [formal language](#) that interacts with the computer can be seen as a programming language. It is not important how this interaction happens. It can be for example graphical, textual or a mixture of both or another way that the user communicates with the computer. The intention of a programming language is to provide a [framework](#) for interaction and not the [Graphical User Interface \(GUI\)](#). How the GUI actually looks like is of no concern for the programming language. In other words: you can't see the programming language. It is about what you can do with it. In architecture the GUI would be the real built building and programming languages are the building technologies. Without building technologies it is impossible to build a building.

Of course the GUI is very important for the user and should be integrally thought with the features but these systems should stay separate. This reason why the GUI is important is the same why future users should be taken into account when designing a building for them. They will be the one using it and so they have to be comfortable with it. This of course only applies if you want to create user friendly software or architecture. Every feature that is built into a programming language and is not used, is a dead feature and makes the programming harder to learn because there is more ballast attached in the documentation.

The reason why GUI and the programming language should be separately implemented is comparable to why architectural plans should have scale. [2](#) Just like in a planing process in architecture the complexity of software development rises with time. The difference is that a software never stops changing and an architectural planing stops, once the building is built. Unmaintained software has soon security issues and the platform they run on also stops existing at a point. It is crucial that the software is written with good abstraction levels. A programming language is one way to set such an abstraction level.

The purpose of SemIO is to ensure a project specific digital [framework](#). Unlike [IFC](#) which "is a standardized, digital description of the built environment, including

buildings and civil infrastructure.”¹ Never the less SemIO is similar to IFC because it also defines the abstraction pattern behind a project but it operates on top of other frameworks. Any type can be referenced. Therefore SemIO is not about replacing any data exchange format but about glueing existing ones together.

3.1 Paradigms

The following paradigms are the foundation of SemIO.²

3.1.1 High-level

High-level programming languages are programming languages with a high abstraction level. In the introduction I mentioned that a computer has many programming languages that are chained one after each other. ¹ The highest is the one that humans use to interact with the computer. As soon as a program gets compiled it loses more and more of readability for humans but it becomes "more" understandable for computers. The last level where the computer actually computes something is the lowest level programming language. The code of this programming language is called *machine code*. One question that might appear, why do you need another high-level programming language if there are already big ones available? In the end you want computers to take over a specific task for you and this task is located inside a specific domain. The closer you can communicate with the computer, the more the computer can help you.

CAD is one example. *CAD* takes care that you don't have to redraw drawings all the time you make a change. *How* *CAD* really "draws" is unimportant to the user. As a user you just want to be able to tell the computer the important things: The line width, the line pattern, the line color, etc . The more abstract you can program, the faster you can use the computer to do something for you. Besides that you can create faster products and you need less knowledge about computers and you can focus instead on learning new knowledge from your domain. An architect doesn't need to know about all different types of tools of a carpenter and how to use them. As long as he knows what a carpenter can do with them he can design a project that can only be build by a carpenter. I am not arguing that it doesn't help to know details of actual processes but just that the knowledge is not necessary. The difference is that without high-level program interfaces architects can simply not design with computers if they are not programmers at the same time.

Frampton et al. (2009) summarizes further: “ The power of high-level languages lies in their abstraction over hardware and software complexity, leading to greater security, better reliability, and lower development costs. ” To explain that I would have to go much further and that would be out of the scope of this thesis. The reasons behind that are also not important for architects who intend to use this programming language.

¹*buildingSMART International, Ltd (2020 b)*

²I am not only referring to programming language paradigms but to paradigms in general.

3.1.2 Domain-specific

The term domain-specific already implies that this programming language is build for a specific domain: Architecture. More specific it is build for architects. The paradigm of high-level is bond very much to the paradigm domain-specific because only a high-level code has the [expressivity](#) that a [Domain Specific Language \(DSL\)](#) needs. DSL are the opposite to [General-Purpose Language \(GPL\)](#)³. A GPL gives the tools to build *anything*. A commonly used language is for example Python. The [Python Software Foundation \(2020\)](#) explains their language as following: “Python is a high-level general-purpose programming language that can be applied to many different classes of problems.” In order to solve a wider spectrum of problems with the same language you need to provide enough symbols that can describe your specific problem.

Let me give you an example. Imagine you go to the library because you are looking for [plans](#) of a project. You ask the library staff: "Do you have *plans* of that project?". If you are lucky they recognized that you are an architect and they also know what architectural plans are. Therefore they could guess that you mean: floor plans, sections, etc. . If you are unlucky they think you are an economist and that you want the system investment plan, etc of this project. If that library would be limited to architecture and they only had staff who know about architecture, this would reduce the risk of misunderstanding significantly. Limiting possible meanings allows you to speak more expressive⁴ and if your library only contains architectural books then why make conversation unnecessary complex?

In this example it doesn't seem like a big deal to spare some words in these sentences but for programming to be able to write more abstract code is very important. Some of the consequences were described in the subchapter above ([High-level](#)). According to ([Spinellis 2001](#), p. 92-93) and [Van Deursen & Klint \(2002\)](#) writing programs in DSL instead of GPL have following advantages:

Advantage

Direct involvement of the domain expert

Expressiveness

Runtime efficiency

Modest implementation cost

Reliability

Tool support limitations

Training costs

Design experience

Software process integration

Reduced maintenance costs

Higher portability

Optimizability

Testability

³The acronym GPL can also be used for General Public License which can be confusing

⁴Expressive doesn't mean describing a thing with more words but describing a thing with less words and still say the same. See [expressivity](#)

3.1.3 Object-oriented and functional

Object oriented (OO) and functional are both programming paradigms. In computer science objects are used to form abstract models. In architecture model are used to represent graspable objects. Even if architectural models are digitally represented, they will remain virtually graspable due to their form. OO programming is therefore necessary to support architectural thinking. The creation of abstract objects in the understanding of computer science is exactly what I want to introduce with SemIO.

It is not intuitive why the *functional* paradigm is necessary for this programming language. In order to allow procedural modelling, *functions* that represent rules are very important. To be able to exchange logic inside the programming language, it is necessary to support the functional paradigm. Here I should mention the only other DSL in regards to designing architecture that I know: Computer Generated Architecture (CGA) from Müller et al. (2006).⁵ CGA is very powerful *functional* DSL for rule based *visual* production of architectural images and videos. To not have to model cities manually and create entire building models, but only produce the part that you see, is crucial for animation studios. Therefore CGA is mainly build for creating masses and textures. Most buildings that are produced in CGA only look like buildings but they are actually just dummies. It is possible to create more detailed massing until a certain point but that is not the intention of the software CityEngine where CGA is embeded in. Esri (2020) advertises CityEngine with: “Build entire 3D cities”. The focus is clearly to BIM support urban planing and animation and not extensive BIM support for generating building schedules, documentation etc. Nevertheless the potential of CGA is enormous because once CGA gets linked to rules based architectural modelling instead of dummies this can become a powerful tool to link different abstraction levels.⁶

Another heavy limitation of CGA is that *shape grammars* are geometrically defined. This is no wonder, because the concept of *shape grammars* was introduced years ago by Stiny & Gips (1971) and they wanted to provide a formal language for geometry. This limitation also applies to procedural modelling tools like Grasshopper from Robert McNeel & Associates (2020) and Dynamo from Autodesk, Inc (2020).

To support shape grammars and procedural modelling it is therefore crucial to allow *functional* programming.

3.1.4 Compiled

There are two ways that programs can be understood by computers: interpreted or compiled. They do basically the same but they way they do it is different. To understand how they do lets take a look at what code really is. Code is nothing else than a series of different characters. Characters are alphabetical symbols, numbers, whitespaces, additional symbols like brackets ()[]{} , etc. A programming language will define what syntactically correct code is. In order to find out if your program

⁵There is another DSL in the field of AEC: Building Environment Rule and Analysis(BERA) by Lee et al. (2014). As the focus of BERA is “flexible design review tasks especially for retrieving and computing specific dataset ” (p. 1), I didn’t count it as a *design* tool but as an *evaluation* tool.

⁶Esri had 2012-2014 according to A.3 the highest global revenue in the spacial information analytic tools software segment.

performs as you want it to perform, there is no other way than testing it.⁷ This process is called *debugging*.⁸ If you have many lines of code and you change some small things and the new code has some parts that are syntactically wrong, it can be helpful if the computer still lets you run the code until the point where it will eventually break. Maybe the machine will not even reach that point of the program where the wrong code is. An interpreter would allow you to run this but a compiler wouldn't. The reason behind this is that an interpreter only translates command after command and the compiler translates everything in once. If there is one mistake in the code the compiler will not compile anything until all syntactical problems are solved. But an advantage of compiling is that it can optimize the code along the way of reading it because it knows the whole program. An interpreter doesn't know what is coming and can therefore not optimize the way a compiler can.

Let me give an example from architecture. Imagine the code which is written in SemIO represents all **plans** of a building and when the code is translated to the computer, this is the equivalent to that the building was built. Compiling would be first checking all plans and see if there is any mistake. If there is a mistake building doesn't start. Otherwise even if the plans start with the upper floors the compiler will try to look for a ground floor first and give the plans in the right order to the builders. Interpreting would be to just start with the first plan, give that to the construction firm regardless if that is the first floor because the interpreter can not go through the stack of plans and search for the right one. Only if plans are in correct order this no problem.

As SemIO is mainly about abstractions of information it allows you to create your own objects and reference them to other objects. If SemIO would be interpreted, all these commands would have to be declared before used. Only one order would work. But when SemIO is compiled, the compiler can figure out what it is referring to what, even if something gets defined later. The order doesn't matter any more.

3.1.5 Open-source

The concept *open-source* literally just says that the source of software has been opened. What does that mean? First you need to understand what the *source* is. Source describes the source code of a program. I showed earlier that when **code** is written it gets compiled many times before the computer executes it. **1** Every time a **compiler** translates code it becomes less and less readable for humans. In the subsection above I explained that code is just a series of characters. **3.1.4** A computer translates every character into a binary number. When you run a program it doesn't need to be readable for humans any more. Therefore if you are done writing your source code and your software works fine, you ship the so called *binary* of it. The binary is the compiled version of your source code. An example to understand this is the naming of variables. Variables are containers that store information. When you

⁷You can't know beforehand if a program will solve a problem and stop, without running this program. The mathematical problem behind this is called the *Entscheidungsproblem* which was formulated by Hilbert. It was mathematically proven by Alan Turing that the: "Hilbert Entscheidungsproblem can have no solution." (Turing 1937, p. 259)

⁸The reason why unexpected behaviour is called a *bug* is fairly simple because in early days of programming, machines were huge and when a bug flew into the electrical circuit it could cause unexpected behaviour. Therefore you had to sometimes look for bugs. This expression stays until today.

program you give these variables a name. To represent the age of a person you can call your variable "ageOfPerson". What "ageOfPerson" actually stores is a number. You could also have named it "number1". The etiquette for your container doesn't matter for the computer. Without expressive names a human can't program. During the compilation process the computer will replace all variable names with something shorter.⁹ Even if your code still does exactly the same, without any name telling what is being done, it is really hard to guess why this program behaves in that way. This process of changing etiquettes is actually something useful if you want to sell your program. This makes it harder for someone to steal your code and modify it for his purposes. Being able to "steal", modify and redistribute the program for free is the idea of *open-source*. Open-source stands for this idea and is not a specific license. In fact there are many different open-source licenses. Central questions are: Once you modified the code, can you suddenly sell it as your own or does it have to remain free? Do you have to document and publish what you changed? There is the full spectrum from no restrictions at all¹⁰ until very restrictive¹¹

A core function of SemIO is to act as an interface between architects and other involved planers. Therefore it is crucial that SemIO is accessible for everyone. For example the [German Federal Ministry of Transport and Digital Infrastructure \(2015\)](#) demands in the “ Conditions for digital design, construction and operation ” (p. 4) that

For the collaborative work with BIM, the data exchanged between the participants have to be compatible. Therefore, it is indispensable for making full use of BIM that all software manufacturers use the same standardized and non-proprietary exchange formats and structural component descriptions. (p. 4)

Open-source is a very effective tool to guarantee that. Further it allows international communication and participation of governments and individuals.

3.2 Concepts

This section is the core of SemIO. In the following I will explain the main concepts behind this programming language. The concepts will be directly used by the user later in order to abstract a project. Every architectural [project](#) needs to be represented with these concepts. They provide the freedom and limit of a project that is designed with SemIO. From here on all concepts that are used in the titles are new [concepts](#) for SemIO and do not refer to the common meaning.¹²

⁹This is simplified and depends on the actual compiler and the programming language.

¹⁰The MIT license is an example for that where you just have to quote the name. There is no further restriction. The license text can be found at: [Open Source Initiative \(2020\)](#)

¹¹The most common one is the General public license. [Free Software Foundation \(2020\)](#)

¹²To underline that a [concept](#) is used with a new meaning, it will be marked with [purple](#). They are not linked and therefore not clickable. The colour is used purely for notational purpose.



3.2.1 Project

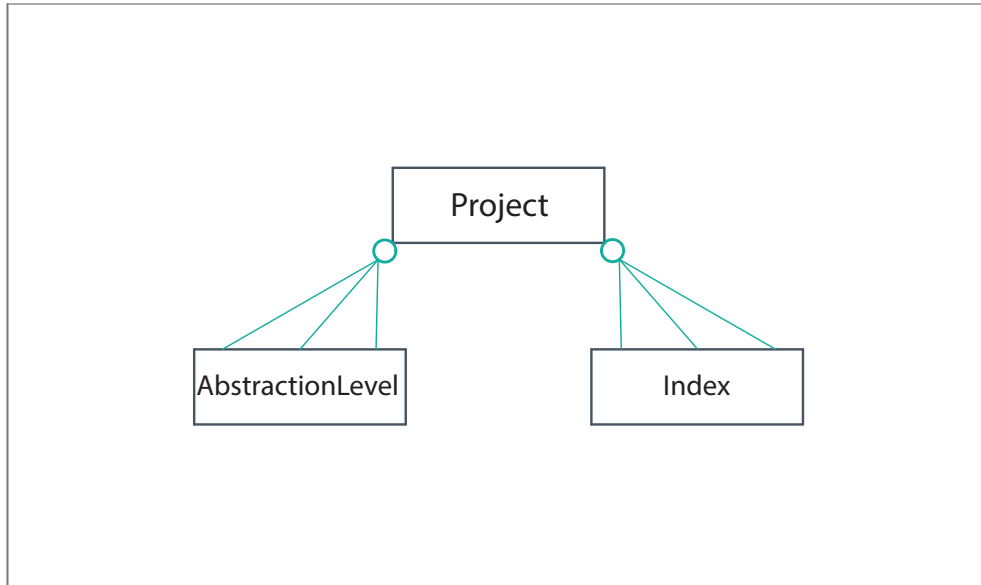


Figure 3.3: Project diagram

A **project** is the container that contains all necessary information about the planing of an architectural project. The main elements of a **project** are **abstraction levels** that contain the project information and **indices** that evaluate the overall performance of the project. Performance is arbitrarily definable but i'll cover more about that in the subsection **Indices**.

3.2.2 Abstraction levels

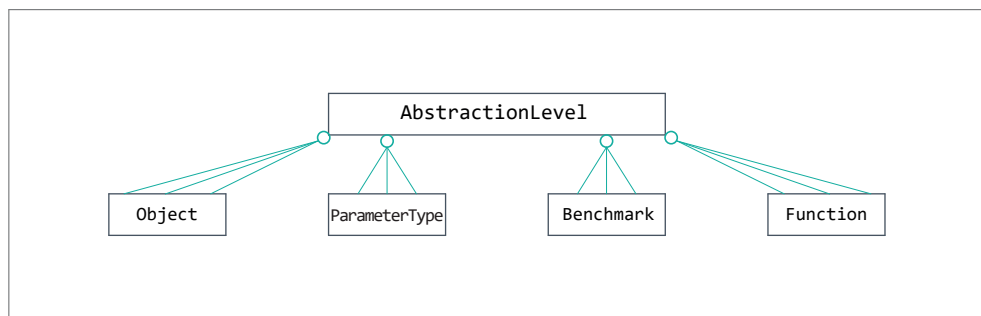


Figure 3.4: Abstraction level diagram

An **abstraction level** consists of **things** , **parameter types** , **benchmarks** and **functions** . All these sub-concepts are strictly tied to the **abstraction level** and

can not exist on their own.¹³ The reason behind this restriction is to ensure that **things** only contain *adequate* information. In the first chapter I covered what *adequate* informations are. **2**

The main purpose of **Abstraction levels** is to order information. This helps architects to make *adequate* design decisions and better communicate with other architects.¹⁴ **Abstraction levels** are the digital concept for scale. Now I will explain how SemIO allows to design digitally and still support abstraction levels. **Carpo (2017)** uses an original motto from Gmail: “Search don’t sort.” (p. 23) and explains with that a new paradigm shift in modern science. Sorting would be to create a box¹⁵ for everything and pack boxes into other boxes until all things are stored. The opposite would be to label every item with certain etiquettes and put them all in the same box. This way of storing data only works if you have an efficient *searching* algorithm. For SemIO this means: Should abstraction level be boxes or labels?

Technically both ways work for creating digital things but the labelling version is way more free and less complex in order to create new categories. In computer science it is actually a good practice to implement things in the way of labelling.¹⁶ But for designing architecture this freedom is exactly a problem. When you design you have to make decisions and you have to find a way to limit options from an unlimited space of possible designs. This is what an **architectural concept** does. Therefore I believe if you want to allow the digital formulation of an architectural concept, you have to allow an efficient way of limiting the design. Commonly in architectural practice this is done over *sorting* by scale.¹⁷ If the labelling needs to be done by the designer, it would actually be the opposite of efficient for the designer because as soon as he creates a new thing he would have to label it. If you design in an abstraction level then you don’t want to label this abstraction level all the time. The only advantage from the users perspective would be that he can label a thing with multiple abstraction levels. If a thing can be part of multiple abstraction levels, then this thing is not precisely abstracted and there is no point in using abstraction levels at all.

The decision to go for boxing instead of labelling has for the technical development some consequences. This means that the **framework** can’t grow larger as it could with labelling because the complexity rises as well. For this programming language this is fine because it is a **DSL**. We saw that the advantage of a DSL is exactly to limit the domain and reduce complexity so the domain users can *program* without having much programming knowledge.

¹³In programming this kind of relation is called composition.

¹⁴The reason behind that was covered in the first chapter. **2**

¹⁵Carpo uses the term *folder* for this.

¹⁶The design pattern behind this is called "Composition over inheritance". The Gang of Four explains the advantages as following: “ Favoring thing composition over class inheritance helps you keep each class encapsulated and focused on one task. Your classes and class hierarchies will remain small and will be less likely to grow into unmanageable monsters. ” (**Gamma et al. 1994**, p. 32) In case these concepts are unknown, I introduced design patterns and the Gang of Four in the introduction. **1**

¹⁷In the first chapter I covered why scale is important for the design process. **2**

Abstraction Levels are to some amount comparable to **Level of Details (LODs)** in the **BIM** context. Geyer et al. (2018) highlights that

The level of development concept [...] formally describes the information accumulation in planning [...]. There are five BIM model stages during the building lifecycle, which are known as the Conceptual Model, Schematic Design Model, Detailed Design Model, Construction Model and As-built Model. [...] However, these definitions focus more on information aggregation than on the support design process, with activities such as the crucial practice of scaling. In particular, scaling back and forth is not foreseen in the information aggregation concept of the level of development. (p. 519)

The main difference is that you can not introduce project specific LODs. This is exactly what SemIO allows with **abstraction levels**. There are many more concepts that have a similar meaning like *level of development*¹⁸, *level of accuracy*¹⁹, *level of scale*²⁰

¹⁸Arup (2012) uses *level of development* in their free BIM maturity measuring tool the category *Level of Development* for 7 categories (structural, mechanical, electrical, public health, facades, geotechnics, lighting)

¹⁹Zahedi & Petzold (2018) uses *level of accuracy* in the context of speaking about a conceptual framework for data exchange between architects and other planers. It describes the accuracy that a simulation program needs.

²⁰Alexander (2002) uses *level of scale* to describe one of 15 properties of life. This is completely unrelated to the other concepts but still covers the importance of scale. Also when I don't follow the aesthetic definition of Alexander.

3.2.3 Things

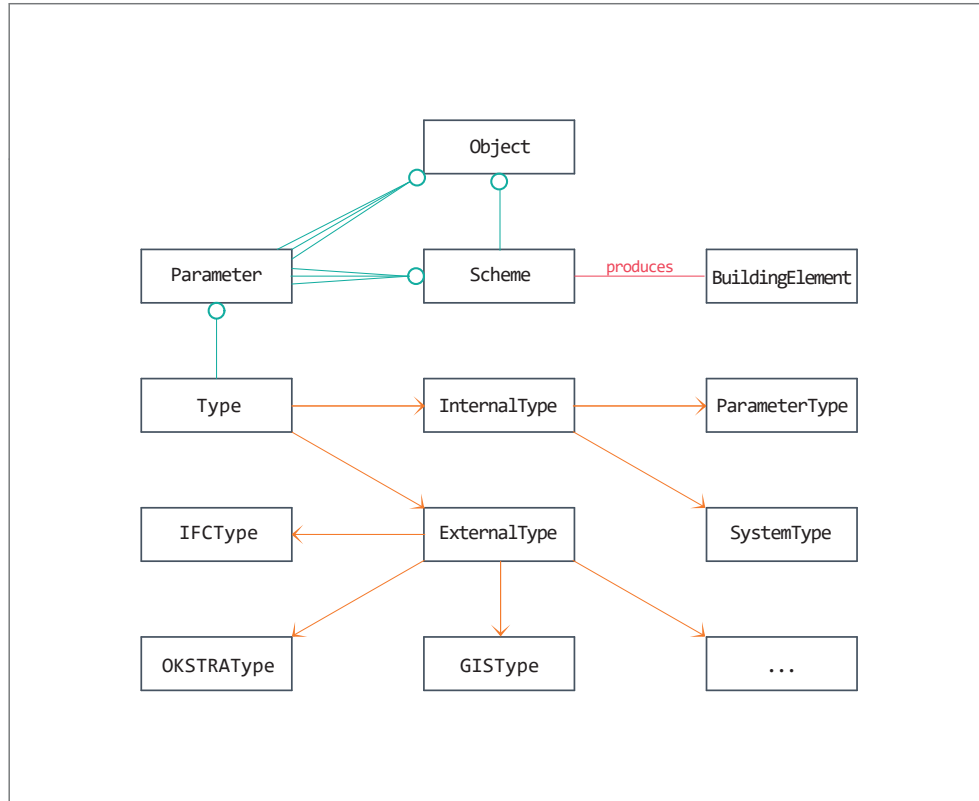


Figure 3.5: Design part diagram

In all subsumptions of an object under a conception, the representation of the object must be homogeneous with the conception; in other words, the conception must contain that which is represented in the object to be subsumed under it. For this is the meaning of the expression: "An object is contained under a conception." Thus the empirical conception of a plate is homogeneous with the pure geometrical conception of a circle, inasmuch as the roundness which is cogitated in the former is intuited in the latter.²¹ p. 76 Kant (1788b)

This conception of an object what Kant describes here happens with the concept **thing**. A **thing** allows to define the *circle* first and then after you can define the derived **thing** *plate*. The hierarchy between both will be kept. This means a

²¹“ In allen Subsumtionen eines Gegenstandes unter einen Begriff muß die Vorstellung des ersteren mit der letzteren gleichartig sein, d.i. der Begriff muß dasjenige enthalten, was in dem darunter zu subsumierenden Gegenstande vorgestellt wird, denn das bedeutet eben der Ausdruck: ein Gegenstand sei unter einem Begriffe enthalten. So hat der empirische Begriff eines Tellers mit dem reinen geometrischen eines Zirkels Gleichartigkeit, indem die Rundung, die in dem ersteren gedacht wird, sich im letzteren anschauen läßt. p. 196-197” Kant (1788a)

thing represents anything.²² This can be something graspable like "house", "wall", "window", ... or something abstract like "sensation", "space quality", "feeling". In order to describe a **thing** you need **parameters** because what you are really creating is a **model** that needs to be measured somehow.

It is important that **thing** is only the definition for what a thing is. In this definition you standardize for your project that a house needs to be represented by a volume and the amount of people who live there. Or that a room is represented by a contour, the entrance point and the actions that can take place. All these definitions together will make sure that all involved planer get the data that they need. As it is explicitly defined what a **thing** is, when later designing you need to provide all the necessary information otherwise you can't create the actual **thing**. The concept that takes care of the actual creation of specific **things** is the **scheme**.

3.2.4 Schemes

Now it is quite clear that there must be some third thing, which on the one side is homogeneous with the category, and with the phenomenon on the other, and so makes the application of the former to the latter possible. This mediating representation must be pure (without any empirical content), and yet must on the one side be intellectual, on the other sensuous. Such a representation is the transcendental schema.²³ p. 76 Kant (1788b)

While the **thing** is the concept for the *category* and the **parameters** describe the *phenomenon*, the **scheme** is the procedural rule that describes *how* a **thing** is produced in general. This *third* is very important theoretically but also practically. In current procedural modelling software like Robert McNeel & Associates (2020) Grasshopper and Autodesk, Inc (2020) Dynamo this *third* exist but only without an explicit **thing** definition.²⁴ Introducing **things** adds another abstraction to the project and therefore interfaces can be set without a *concrete* design. The **scheme** separates the *concrete* from the *abstract*. When architects, engineers and other planers interact they need certain *type* of information. Which *number* it is, is not relevant for the workflow. This separation has big advantages. You can simply exchange one **scheme** with another **scheme** but as they are creating the same **thing**, this will not disturbing the overall workflow. Further this guarantees architects complete freedom for their design as long as they implement the necessary definition of the underling **thing**. The **scheme** can also contain further **scheme**-specific **parameters**. As those

²² *Anything* can be understood literally in the heideggerian sense of any *thing*[Ding] and not only concrete things [Gegenstände]. In the first chapter Bruno Latour refers to Heideggers concept of *thing*. 1

²³“ Nun ist klar, daß es ein Drittes geben müsse, was einerseits mit der Kategorie, andererseits mit der Erscheinung in Gleichartigkeit stehen muß, und die Anwendung der ersteren auf die letzte möglich macht. Diese vermittelnde Vorstellung muß rein (ohne alles Empirische) und doch einerseits intellektuell, andererseits sinnlich sein. Eine solche ist das transzendente Schema. p. 197” Kant (1788a)

²⁴The scripts contain pure logic. *Objects* do not exist. There are only parts of system in an unordered way. The computer is not aware of what an object is and what belongs to it even if they are clustered. The only way to bypass that is to program yourself. Therefore exchanging and designing in a larger team with Grasshopper or Dynamo is very challenging because you have to limit your own boundaries. There is no tool like Software Freedom Conservancy (2020) that helps for versioning and merging of the same parallel files. One reason behind that is that as there are no *objects*, there is no signature to check.

are not necessary, other involved planers can't expect any of them. If a structural engineer needs more data then this needs to be integrated as a **parameter** of the **thing**. This way all involved planers can access relevant data and automatic digital data exchange is ensured.

The **scheme** is responsible for creating actual **building elements**. **Building elements** are therefore not directly tied to **abstraction levels** but indirectly over **things** which link to the **scheme**.

3.2.5 Parameters

In the case of a design problem which is truly problematical, we encounter the same situation. We do not have a field description of the context, and therefore have no intrinsic way of reducing the potentially infinite set of requirements to finite terms. Yet for practical reasons we do need some way of picking a finite set from the infinite set of possible ones.²⁵ p. 26
Alexander (1964)

This *field of description* happens in SemIO over **parameters**. **Parameters** measure **things**. This measurements happens mainly with other concepts and numbers. Concrete **things** will have more numbers like width, height, ... and abstract **things** will be measured with other concepts like "interesting", "boring", "expressive", "intimidating", ... All **parameters** that measure a **thing** are discrete and finite.

Types **Parameters** always have a **type**. Besides the name you give them they are always referring to something else, a value²⁶ of a certain **type**.²⁷ Without a **type** you could just store words with no meaning. In this case the **parameter** would be an empty concept. In fact all the concepts in SemIO are also a **type** but they disappear after the compiling process. So therefore the only relevant **types** are **internal types** and **external types**.

Internal types **Internal types** are for example **Text**, **Number**, **TrueOrFalse** or a **parameterType** that were defined in the **abstraction level**. What exactly is a **parameterType**? A **parameterType** is a set of concepts that describe a type. For example a **parameterType** "activity" could be described with the concepts "sleeping", "relaxing", "working", "cooking". Therefore if the **parameter** with the **parameterType** "activity" appears, it needs to be one of these values. In case you want the empty state to exist, you just add "none" as an option. Further if you don't want to be limited to choose only one, you can say that multiple "activities" can exist

²⁵Alexander solves this way to think from "a negative point of view, as potential misfits, there is a simple way of picking a finite set. This is because it is through misfit that the problem originally brings itself to our attention." (p. 26) The **condition** will allow define such requirements.

²⁶In computerscience you call that value an instance of the class (**type**.)

²⁷Technically that can be miss understood. I don't mean the common definition of reference and value types. Although some parameters (of type struct for example) are directly stored on the heap and are not only a pointer, I mean the fact that any parameter is represented by the name (string).

at the same time. ²⁸

External types An **external types** can be anything from an external **framework**. In the design part diagram 3.5 I gave three examples of possible frameworks. The most important one is **IFC**.²⁹ For example if you wish to create the **parameter** "zone" then you can choose to give it the type "IFCZone".³⁰ Therefore if you don't want to create a new definition for "zone", you can just refer to existing object models. The second example is an object catalogue for road and traffic engineering.³¹ If you want to add a street network to your **abstractionLevel** "masterplan" then you can just refer to the **type** "Strassennetz" [english: street net] from the object catalogue.³² The third example is a standard for exchanging scientific data.³³ Geotechnical or climatological data like "topopgraphy" or "weather" can just be added to "masterplan" referring to the data in the **GISType type**. Those three examples were arbitrary. The only restriction is that you the framework needs to be expressible in a **Document Object Model (DOM)**.

²⁸A **parameterType** in computer science is called an *enumeration*. If multiple values would be allowed at the same time then this is *flagged enumeration*. The reason behind this is because every option is translated into a number. For example "sleeping" would be represented by the number 1, "relaxing" by the number 2, "working" by the number 3, etc. If it is *flagged* it wouldn't be enough to give "sleeping" the number 1 and "relaxing" the number 2 because if the number $1 + 2 = 3$ appears then that would be "working" instead of "sleeping" and "working" together. Therefore if you reserve the number 3 for the combination of "sleeping" and "relaxing" and "working" would suddenly get number 4, then there is no confusion. Therefore in a *flagged enumeration* all concepts are encoded with the value $2^{(\text{the position of the concept})}$.

²⁹[buildingSMART International, Ltd \(2020 b\)](#)

³⁰[buildingSMART International, Ltd \(2020 b\)](#)

³¹It was developed from the [German Federal Highway Research Institute \(2020\)](#).

³²I can't give a direct link to this class because the website is an application and doesn't work over links.

³³This standard is developed by [University Corporation for Atmospheric Research, \(2020\)](#).

3.2.6 Indices

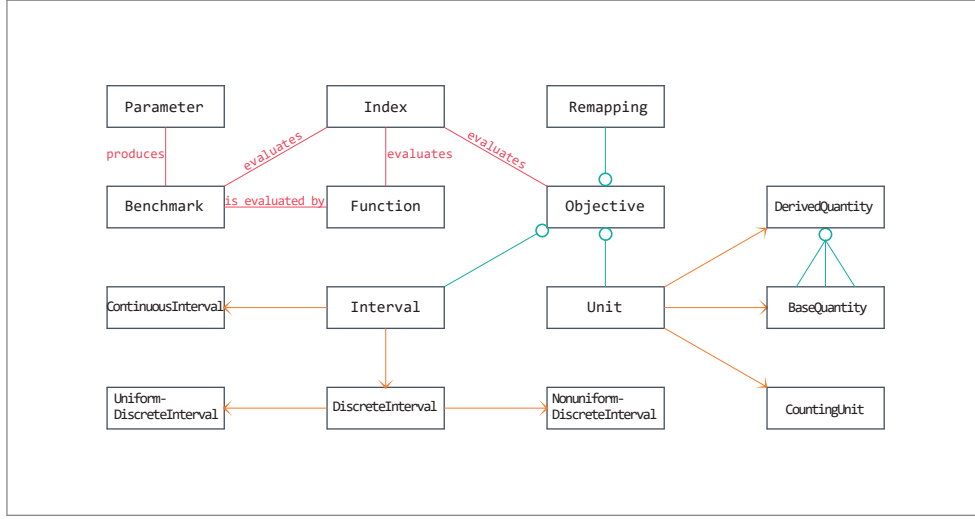


Figure 3.6: Evaluation part diagram

Indices are the second part of SemIO. An **index** will evaluate a certain part of the project. This can be a predefined index like the a index for sustainability³⁴ or a custom **index** that is measuring the amount of triangular edges that appear in the floor plan and evaluates the amount of edges on a custom scale. A triangular edge could be defined as two walls coming together in an angle between 55° and 65° and the number could start with minimum 3 and the maximum of 12. The rewards for a certain amount of edges can be defined by the architect. For example could 9 edges achieve the highest score and from the range 3-9 it could be jumping between lower and higher rewards and from the range 9-12 a linear reward function. Regardless the purpose of the **index**, it will need criteria, to evaluate data and a rewarding mechanism. The data will be provided by **benchmarks** or, if it is not directly computable, **functions** can give the desired result. **Conditions** will set the goals and their evaluation for a certain **abstractionLevel**. Over **remappings** the evaluated data can be mapped to a score. In the **index** the weights of the **conditions** can be adjusted.

3.2.7 Benchmarks

Benchmarks are created by **parameters**. A **parameter** can be tagged as **benchmark** and then a new **benchmark** will be created or the value will be added to an existing **benchmark** with the same name. Whenever an actual **thing** gets created over a **scheme**, the actual data will be created. As the computer knows what this **thing** is, it can automatically ask for that data. For example a **thing** "room" has been created in an **abstractionLevel** "roomplan". The **thing** room contains a "surface" as a **parameter**. "Surface" is already another **thing** that contains **parameters** to

³⁴An example could be the "Bewertungssystem Nachhaltiges Bauen" (BNB) from [German Federal Ministry of the Interior, Building and Community \(2015\)](#)

describe geometrically a surface. The area of this surface is needed by the economist to calculate if the **project** creates enough rentable area. The architect can tag the **parameter** "surface" as a **benchmark** and call it "roomsArea". As the economist doesn't need the whole geometrical **surface** but only a number that represents the area, the **benchmark** room can be limited to only contain the **parameter** "area" that is a **parameter** from the **surface**. Now every time that a scheme produces a **thing** "room" the "area" of the **parameter** **surface** will be added to the global **benchmark** "roomsAreas". The economist doesn't need to ask the architect for areas any more as the **benchmark** "roomsAreas" will automatically update. **This is again an example how the explicit **thing** definition completely changes interaction between the architect and the involved planers.**

3.2.8 Functions

With **benchmarks** we saw how the architect can share live data to other planers. After sharing that data, the architect expects an answer from other planers for important questions. For example: Is the project permitted by the law in this context? What is the cost estimation for this project? Can this design of the buildings structurally work? **Functions** are there to provide the automated answer. Over **functions** other planers can make their expertise digitally available. As soon as all the necessary data is **benchmarked** then other planers can use this **benchmarks** for their calculation purposes. The **functions** are for the other planers what **things** are for the architect. They provide the interface definition for the planing process.

3.2.9 Conditions

With the raw data from **benchmarks** and the qualified data from **functions** finally **conditions** can be defined. A **condition** will measure exactly one aspect of the design. Every **condition** is created inside of an **abstractionLevel**. Each condition needs to be defined either as **necessary** or as **sufficient**. For example for an **abstractionLevel** "masterplan" you could create a **condition** "timeToNextPublicTransportStation" that will evaluate the time it takes from a building to the next public transport station. We see that every **condition** has a **unit**. To give a **condition** a score there is the necessity for a scale. The scale is defined by an **interval**. To not only allow minimize, maximize and a linear scoring system there can be custom **remappings**.

Unit The **unit** can be of 3 different **unit types**. In the first case it just measures the amount of some **things**. Then it is a **countingUnit**. Otherwise it is either way a physical **baseQuantity** like length, mass, time, temperature, luminous intensity or **derivedQuantity**

Intervals The **interval** describes the range of possible values that the **condition** can take. The most undefined interval is a **continuousInterval** where only the two limits must be provided. If the **unit** is a **countingUnit** then we automatically have a **discreteInterval**. In this case you need additionally a step size. Sometimes not all the options are available and the **uniformDiscreteInterval** changes to a **nonuniformDiscreteInterval**. In that case all possible values need to be told explicitly.

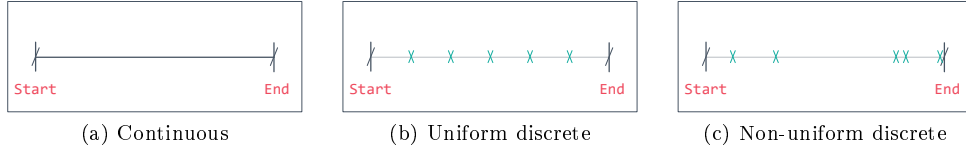


Figure 3.7: Interval diagrams

Remappings The **remappings** make sure to allow more freedom in evaluating **conditions**. The easiest **remapping** is just a linear **remapping** that either way increases or decreases. If it increases it means that the higher the **condition**, the better. If it decreases vice versa. Depending if the starting value or the end value of the **interval** are not the highest and lowest score they are called **biased**. If they both differ for the same amount then the **remapping** is **evenlyBiased** otherwise it is **unevenBiased**.

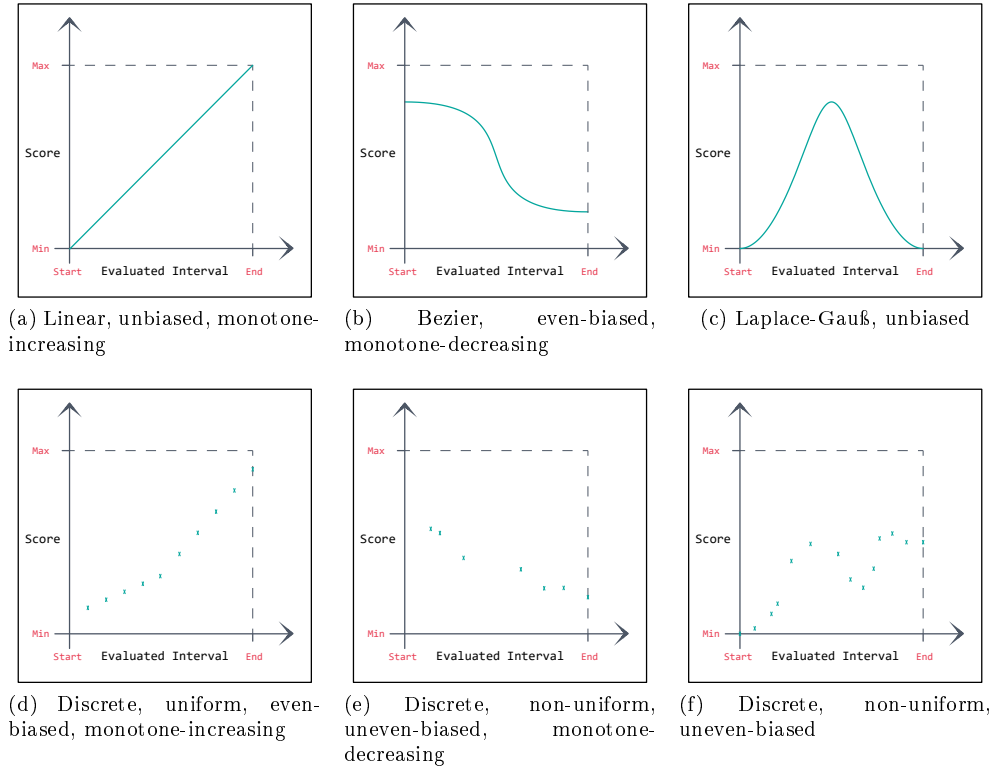


Figure 3.8: Examples of remapping diagrams

4. Prototype

4.1 Syntax

When designing the syntax of SemIO I wanted it to contain as little extra symbols as possible and be as expressive as possible. This is supposed to provide a good learning curve for non-computer scientist and not demanding more technical knowledge then necessary.

Code 4.1: SemIO code Syntax demonstration

```
1 "Your description for the 'NameAbstractionLevel'."
2 You can add to every object a description
3 or leave it empty."
4 AbstractionLevel NameAbstractionLevel
5
6     "If the keyword '[Multiple]' is added after
7     NameParameterType then multiple values
8     can exist at the same time."
9     Parameter NameParameterType [Multiple]
10         "'Value' will be one of the values that
11         the Parameter 'NameParameterType' can take."
12         Value
13
14         AnotherValue
15
16     "'NameThing' is described by Parameters."
17     Thing NameThing
18         "A Parameter is always:
19         Type followed by a Name and
20         optionally a benchmark tag (#)"
21         Text NameParameterText
22
23         "The Type 'Number' is an internal Type."
24         Number NameParameterNumber
25
26         "NameParameterType refers to
27         the Parameter NameParameterType from above."
28         NameParameterType NameNameParameterType
29
30     "'NameDerivedThing' contains all Paramters
31     from the parent Thing 'NameThing' and has
32     additionally its own Parameters."
33     Thing NameDerivedThing (NameThing)
34         "The benchmark adds all values from 'Area'
35         from all the 'NameDerivedThing' to 'AreaThings'.
36         'Area' is a Parameter from 'Surface'.
37         The total 'AreaThings' is measured in sqm."
38         Surface NameParameterSurface #{Area}AreaThings[sqm]
39         Percent NameParameterPercent
40
41 "This AbstractionLevel is derived from the
42 AbstractionLevel above with the name NameAbstractionLevel"
43 AbstractionLevel NameDerivedAbstractionLevel
44 (NameAbstractionLevel)
45     Thing NameThing
46         Solid NameParameterSolid
```

In the example [SemIO code Syntax demonstration](#) we see the concepts of different order. The [AbstractionLevel](#) is a first order concept, [Thing](#) and [Parameter](#) are second order concepts and [types](#) like [Text](#) , [Number](#) , [Surface](#) , [Percent](#) and [Solid](#) . These [types](#) are all [internalTypes](#) . An [externalType](#) could just be referenced with the name as long as the DLL is accessible by the Compiler.

4.1.1 Line indentation

Objects are parsed according their line indentation. Depending on the order of an object there needs to be the same amount of spacing units. One spacing unit is currently either way four spaces or one tab. There fore before the keyword [AbstractionLevel](#) there must be no space, before [Thing](#) and [Parameter](#) there must be one spacing unit and for all the values there must be two spacing units. Descriptions need to start with the same amount of spacing units but can spread over multiple lines without the need to start with spacing units for every line. The comments are everything between two quotation marks " ... ".

4.1.2 Dot notation

Dot notation is used for example in benchmarks to access sub objects or values. They provide an intuitive way of accessing associated data. When [schemes](#) will be implemented dot notation will be used more extensively. In the moment there is no semantic processing of an object with dot notation and only syntactical recognition.

4.1.3 Bracing

Bracing is only where necessary. Round braces () are used to declare inheritance from objects. For example [AbstractionLevel](#) and [Thing](#) are both an inheritable type. Whenever a type declaration or multiplicity of an object is set square brackets [] are used. The only time where curly braces {} used.

4.2 Implementation

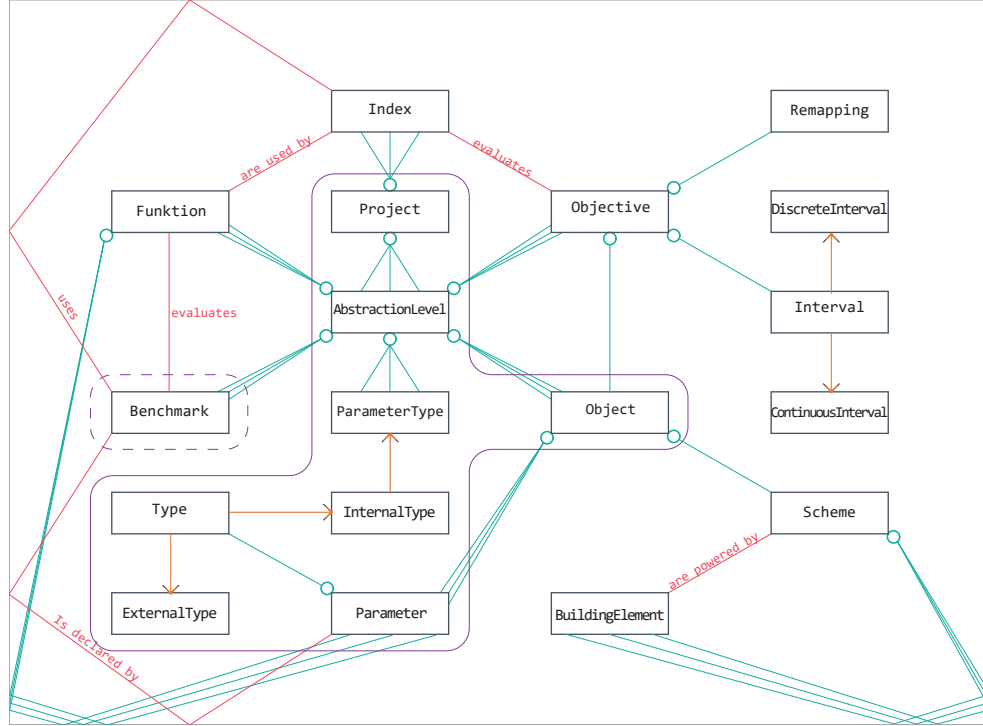


Figure 4.1: Parts of the framework covered by the prototype

The prototype that I implemented into `code` covers the core of the SemIO `framework`. It allows to create a `project` with `abstractionLevels` that contain `things` and `parameterTypes` that can be used as a `type` by the `parameter` that describes the `thing`. Syntactically `benchmarks` are recognized but not semantically processed.

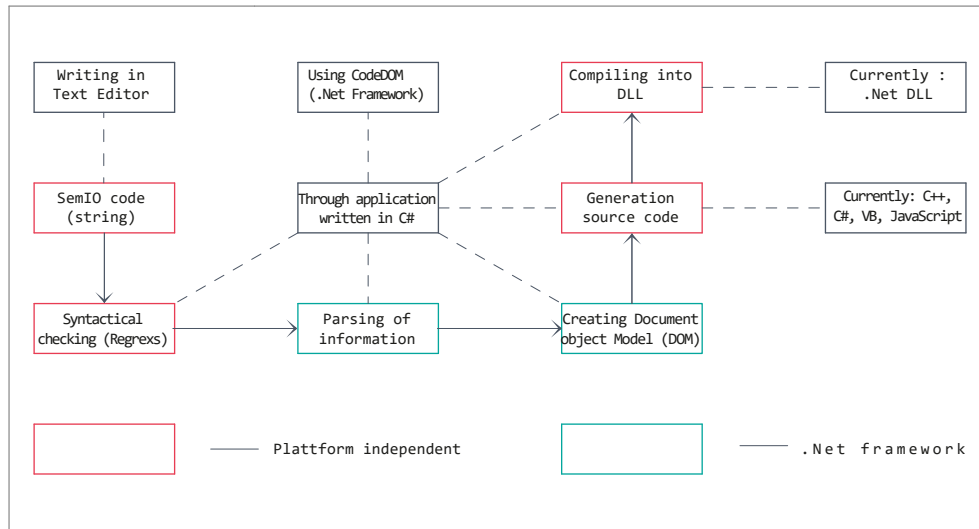


Figure 4.2: Pipeline prototype

In the beginning I mentioned that the **GUI** is not part of the **framework** for a programming language. ³ Therefore the pipeline for the prototype ends where the GUI begins. This prototype is just the **compiler** for the programming language. The input of a compiler is plain text that just consists of a series of characters¹ and the output of a compiler is a file that can be used by other software. In my case this is a **Dynamic Linked Library (DLL)**. Now I will explain how the compiler achieves that. First you need to write a file that is understood by the compiler. After that the text file will be parsed by the **parser** that extracts all the information and removes unnecessary code parts like white spaces. This information is then processed and gets translated into a **DOM**. This DOM is used to generate source code of an **object oriented** programming language. This source code can then be compiled into a **DLL**. The DLL can be used by any software. The complete compiler was written in **C#** which is a programming language developed by **Microsoft (2020a)** that uses the .Net framework². Therefore the compiled DLL is a .Net DLL and can therefore only be targeted by .Net applications. As the compiled code contains only types from .Net core, the supported systems are Windows, Mac and Linux. Detailed information about the code of the SemIO compiler can be found in the open-source online repository.³

4.2.1 Regular expressions

I used regular expressions for syntax recognition and parsing. This is a "hard-coded" approach that doesn't offer great scalability but it allows to stay completely in one platform and was therefore very efficient for my purpose. I didn't have to take care of any additional interface. Therefore a rewrite in a tool like ANTLR from **Parr (2014)** is unavoidable as soon as SemIO is growing.

¹In computer science this text is called a *string*.

²**Microsoft (2020c)**

³SemIO: <https://github.com/usaluz/SemIO>

4.2.2 Document object model

A **DOM** is a platform independent way of storing objects. It can be used as an interface to exchange data or like in my case used as generation tool for source code. The only requirement is that this **object oriented** programming languages provides an DOM interface.

4.2.3 Dynamic linked library

A Dynamic Linked Library is a library for programs. It allows to reference parts of a system to other existing systems. DLLs are not tied to a programming language but they are the compiled result of such. Therefore they target a framework and it is irrelevant how they have been created.

4.2.4 Design patterns

Following design patterns were implemented into the compiler

Design pattern	Author
Lexical processing	(Spinellis 2001 , p. 94-95)
Source-to-source transformation	(Spinellis 2001 , p. 96)
Piggyback pattern	(Spinellis 2001 , p. 93-94)
Task automation	(Mernik et al. 2005 , p. 323)

Lexical processing pattern: I used string processing over regexs.

Source-to-source transformation pattern: With the **DOM** SemIO source code gets compiled into source code of another object oriented programming language.

Piggyback pattern: **InternalTypes** are referenced to the system types and **externalTypes** are referenced to other frameworks. The piggyback pattern is responsible that glueing of existing frameworks happens efficiently with SemIO.

Task automation: All the **thing** and **parameter** definitions would usually have to be defined manually with classes, fields and properties and would have to put into the right namespaces. All this process happens with a massively reduced workload. In the example, [SemIO code Syntax demonstration](#), we see that 42 lines of SemIO code are compiled into 350 lines of C++ code. See [Generated C++ code from compiled SemIO code](#)

4.3 Example

In the following a longer example will be shown. It consists of three **abstractionLevels**: *Masterplan*, *Apartmentplan* and *Roomplan*.

The compiled code will not be provided as the code is around 900 lines of C++ or 650 lines of C# code.

Code 4.2: SemIO code example

```

1 "Massing studies and infrastructure including public
  transportation (Simillar to 1to500)"
2 AbstractionLevel Masterplan
3
4 "Defines the designated use of a parcel"
5 Parameter Use
6     "Mainly factories in the landscape"
7     Industrial
8     Residential
9     "Building areas where buildings are touching and
      streets are formed by enclosing buildings"
10    Urban
11    Mixed
12
13 "Describes an emotion regarding a building"
14 Parameter Emotions[ Multiple]
15     Ecstasy
16     Admiration
17     Terror
18     Amazement
19     Grief
20     Loathing
21     Rage
22     Vigilance
23
24 "Describes a volume with a use"
25 Thing BuildingVolume
26     ClosedCurve Parcel
27     Use Use #Use[dgd]
28     Number Height
29     Emotions Reactions
30
31 "Describes adjacent building volumes"
32 Thing BuildingBlock
33     BuildingVolume[x] BuildingVolumes # {Parcel.Area *
      Height} Volume[cm]
34
35 "General information about layout and accessibility of the
  apartment equivalent to 1to200 in analogue scale"
36 AbstractionLevel Apartmentplan(Masterplan)
37
38 "Describing how an aprartment can be reached with"
39 Parameter Accessibility
40     Walking
41     Rollator
42     Wheelchair
43
44

```

```
45 "Describing how much noise will be emitted from the room"
46 "
47 Parameter NoiseLevel
48     "under 20 dB"
49     VeryQuiet
50     "20 - 40 dB"
51     Quiet
52     "40 - 60 dB"
53     ConversationLevel
54     "over 60 dB"
55     Loud
56
57 "Most general information about an apartment"
58 Thing Apartment
59     "Only one storey footprint of apartment."
60     ClosedCurve Boundary #{Area} AreaLiving[sqm]
61     Point Entrance
62     Integer CountPeople #AmountHabitants[habitants]
63     Accessibility Accessibility #Accessibility
64     NoiseLevel ToleratedNoise
65
66 "General information about how the building is accessed"
67 Thing Circulation
68     ClosedCurve Boundary
69     Point[1<=x] MainEntrances
70     ClosedCurve Staircase
71
72 "Information about rooms, relations between them and openings
73 (windows and doors). Comparable to 1to100"
74 AbstractionLevel Roomplan(Apartmentplan)
75 "Generic base class for openings"
76 Thing Opening
77     Point Center
78     Number Width
79     Number Height
80
81 "Abstract window definition"
82 Thing Window(Opening)
83     Number ParapetHeight
84
85 "Direction of door wing"
86 Parameter DoorDirection
87     Inside
88     Outside
89
90 "Abstract door definition"
91 Thing Door(Opening)
92     DoorDirection DoorDirection
```

```
92 "Activities that happen in a room"
93 Parameter Activities [Multiple]
94     Sleeping
95     Relaxing
96     Working
97     Cooking
98     Showering
99     Bathing
100
101 Parameter StimulatedSenses [Multiple]
102     Vision
103     Hearing
104     Touch
105     Smell
106     Taste
107     Vestibular
108
109 "Intensity of a sense"
110 Parameter IntensitySensation
111     Unnoticeable
112     WeaklyNoticeable
113     Noticeable
114     StronglyNoticeable
115     Unstandable
116
117 "Describes the sensation while being in the room"
118 Thing Sensation
119     AffectedSenses Senses
120     IntensitySensation[x] Intensities
121
122 "Abstract definition of a room with doors and windows"
123 Thing Room
124     ClosedCurve Boundary
125     Window[1<=x] Windows
126     Door[1<=y < 3] Doors
127     Activities Activities
128     Sensation Sensation
```

5. Conclusion

After pointing out the limitations of current digital design software, a proposition for a new conceptual notation method was made. The design and a successful prototype for a new domain-specific programming language could be shown. The possibilities of such a high-level programming language are promising because it introduces the early architectural design practice into a digital environment. This opens many interfaces to existing digital frameworks which couldn't be used before. Further transparency, efficiency and quality can be raised if automated data exchange can happen over an interface definition like a Dynamic Linked Library. Additionally the theoretical and practical dimension that a programming language combines could be partially covered. By targeting directly architects as users, more comfortability in designing digitally is created.

Due to the limitation of the time only the core of the SemIO framework could be developed. The implementation of the prototype allowed to demonstrate the functionality of a compiler that can generate working code. Four programming languages could be generated which offers already high flexibility in the platform and software choice.

The next important step would be to introduce schemes and benchmarks in order to give tools for designing. In the current state SemIO only generates an interface definition. It is not visible yet, how SemIO influences the design process and it would be crucial to see how SemIO could be integrated into the design process. Further a Graphical User Interface could help to improve user friendliness. Early test with future users would show if the tool offers the intended practicability or if the knowledge barrier is too high.

Eidesstattliche Erklärung

Ich versichere an Eides Statt durch meine eigenhändige Unterschrift, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe angefertigt habe. Alle Stellen, die wörtlich oder dem Sinn nach auf Publikationen oder Vorträgen anderer Autoren beruhen, sind als solche kenntlich gemacht. Ich versichere außerdem, dass ich keine andere als die angegebene Literatur verwendet habe. Diese Versicherung bezieht sich auch auf alle in der Arbeit enthaltenen Zeichnungen, Skizzen, bildlichen Darstellungen und dergleichen.

Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Berlin , den 07. November 2020

Ueli Saluz

Acknowledgements

First I would like to thank my supervisor Raoul Bunschoten who always believed in me and would allow me to write such a cross-disciplinary thesis. Without the great freedom that he gave me this work couldn't exist in this context. I am very thankful for that possibility.

My next thank goes to my co-supervisor Philipp Geyer who supported this thesis from the beginning. Having his second perspective was something very valuable for me.

Further I have to thank my family for giving me such great support during this intensive time. Last but not least one person has been there for me everyday and has given me all the support that I needed, my girlfriend Dani. I promise to you that I will cook for you after this!

A. Appendix

A.1 Statistics

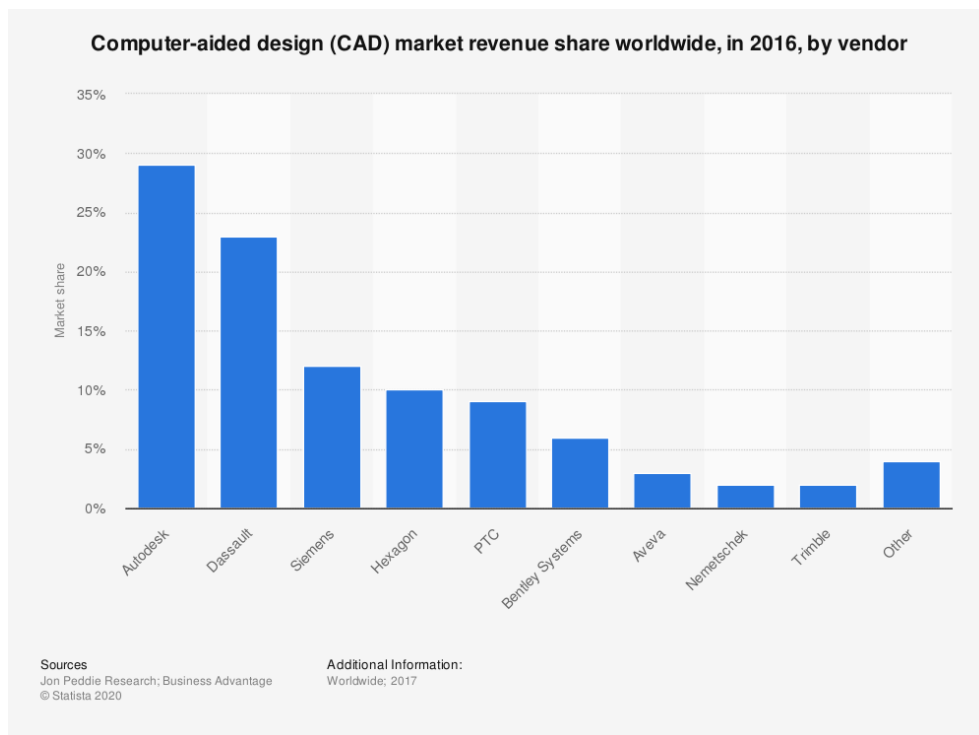


Figure A.1: *cad-market-revenue-share-worldwide-2016*.

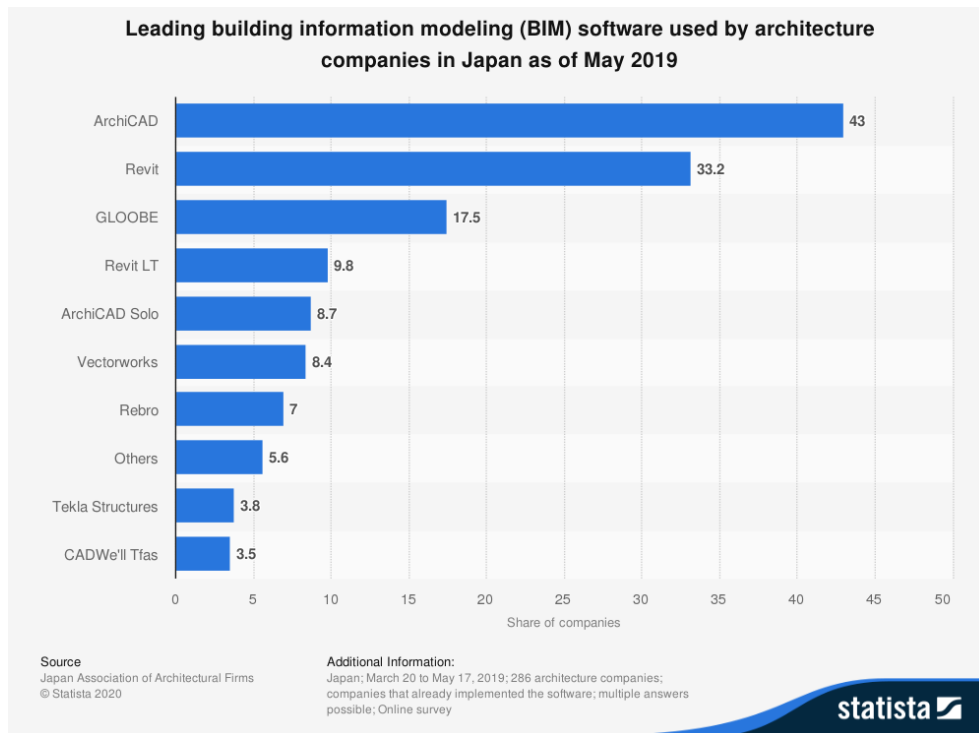


Figure A.2: *bim-software-used-by-architectural-companies-japan-2019*

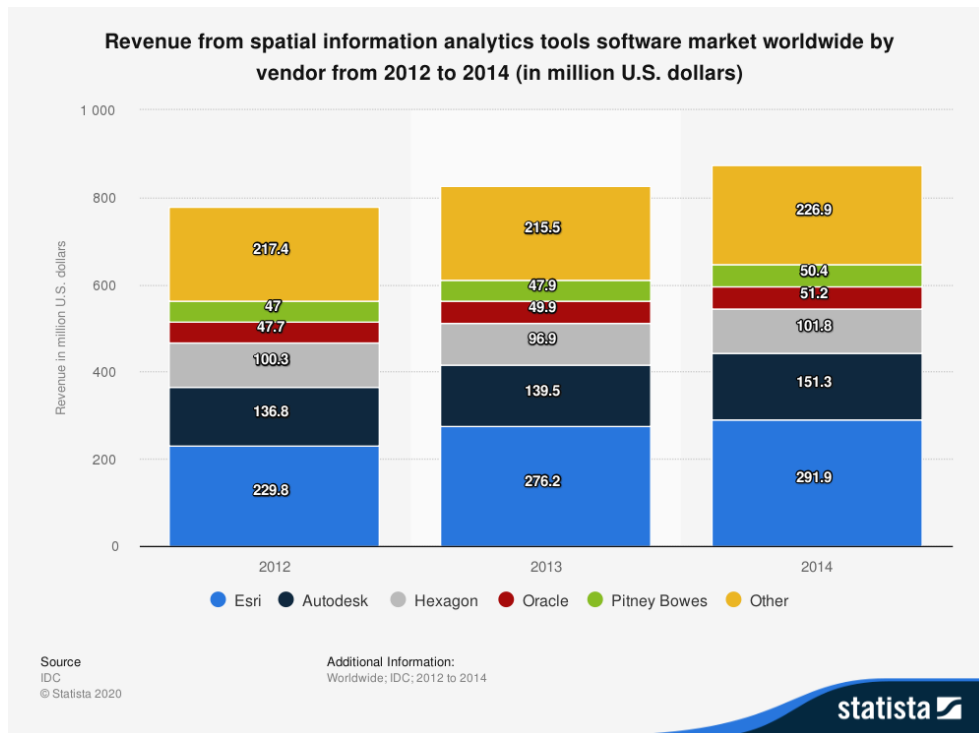


Figure A.3: *global-spatial-information-analytics-tools-software-revenue-by-vendor-2012-2014.*

A.2 Compiled code

Code A.1: Generated C++ code from compiled SemIO code

```
1  #pragma once
2
3  #using <mscorlib.dll>
4
5  using namespace System::Security::Permissions;
6  [assembly:SecurityPermissionAttribute(SecurityAction::
7      RequestMinimum, SkipVerification=false)];
8  namespace AbstractionLevels {
9      namespace NameAbstractionLevelRepository {
10         namespace ParameterTypes {
11         }
12     }
13     namespace AbstractionLevels {
14         namespace NameAbstractionLevelRepository {
15             namespace ThingTypes {
16                 ref class NameThing;
17                 ref class NameDerivedThing;
18             }
19         }
20     }
21     namespace AbstractionLevels {
22         namespace NameAbstractionLevelRepository {
23             ref class NameAbstractionLevel;
24         }
25     }
26     namespace AbstractionLevels {
27         namespace NameDerivedAbstractionLevelRepository {
28             namespace ParameterTypes {
29             }
30         }
31     }
32     namespace AbstractionLevels {
33         namespace NameDerivedAbstractionLevelRepository {
34             namespace ThingTypes {
35                 ref class NameThing;
36             }
37         }
38     }
39     namespace AbstractionLevels {
40         namespace NameDerivedAbstractionLevelRepository {
41             ref class NameDerivedAbstractionLevel;
42         }
43     }
44     namespace AbstractionLevels {
45         namespace NameAbstractionLevelRepository {
46             namespace ParameterTypes {
47                 using namespace System;
48                 using namespace System;
```

```

49
50
51     [System::ComponentModel::Description(L"If_the_keyword_\'[
        Multiple]\'_is_added_after\r\n\tNameParameterType_then_
        multiple_valu"
52 L"es\r\n\tcan_exist_at_the_same_time."),
53     Flags]
54     public enum class NameParameterType
55     {
56
57         [System::ComponentModel::Description(L"'Value\'_will_be_
            one_of_the_values_that\r\n\t\tthe_Parameter_\'
            NameParameterType\'_can_t"
58 L"ake.")]
59         Value = 1,
60
61         [System::ComponentModel::Description(L"")]
62         AnotherValue = 2,
63     };
64     }
65 }
66 }
67 namespace AbstractionLevels {
68     namespace NameAbstractionLevelRepository {
69         namespace ParameterTypes {
70
71         }
72     }
73 }
74 namespace AbstractionLevels {
75     namespace NameAbstractionLevelRepository {
76         namespace ThingTypes {
77             using namespace System;
78             using namespace AbstractionLevels::
                NameAbstractionLevelRepository::ParameterTypes;
79             using namespace SemIOLibrary::Parameters;
80             using namespace SemIOLibrary::Parameters::GeometryParameters;
81             using namespace SemIOLibrary::Parameters::GeometryParameters::
                Surfaces;
82             using namespace SemIOLibrary::Parameters::GeometryParameters::
                Solids;
83             using namespace SemIOLibrary::Parameters::GeometryParameters::
                Points;
84             using namespace SemIOLibrary::Parameters::GeometryParameters::
                Curves;
85             using namespace SemIOLibrary::Parameters::BaseParameters;
86             using namespace SemIOLibrary::Parameters::BaseParameters::
                Numbers;
87             using namespace System;
88             ref class NameThing;
89             ref class NameDerivedThing;
90
91

```

```

92     [System::ComponentModel::Description(L"\ 'NameThing\ 'is _
        described_by_Parameters." )]
93     public ref class NameThing
94     {
95
96         private: Text^    _nameParameterText;
97
98         private: System::Number^    _nameParameterNumber;
99
100        private: AbstractionLevels::NameAbstractionLevelRepository
            ::ParameterTypes::NameParameterType
            _nameNameParameterType;
101
102        public: [System::ComponentModel::Description(L"A_Parameter
            _is_always:\r\n\t\tType_followed_by_a_Name_and\r\n\t\t
            toptionally_a_benchmark_t"
103 L"ag_(#)")]
104        property Text^    NameParameterText
105        {
106            Text^    get();
107            System::Void set (Text^    value);
108        }
109
110        public: [System::ComponentModel::Description(L"The_Type_\ '
            Number_\ 'is_an_internal_Type." )]
111        property System::Number^    NameParameterNumber
112        {
113            System::Number^    get();
114            System::Void set (System::Number^    value);
115        }
116
117        public: [System::ComponentModel::Description(L"
            NameParameterType_refers_to\r\n\t\tthe_Parameter_
            NameParameterType_from_above." )]
118        property AbstractionLevels::NameAbstractionLevelRepository
            ::ParameterTypes::NameParameterType
            NameNameParameterType
119        {
120            AbstractionLevels::NameAbstractionLevelRepository::
            ParameterTypes::NameParameterType get();
121            System::Void set (AbstractionLevels::
            NameAbstractionLevelRepository::ParameterTypes::
            NameParameterType value);
122        }
123    };
124
125    [System::ComponentModel::Description(L"\ 'NameDerivedThing\ '
        contains_all_Parameters_\r\n\t\tfrom_the_parent_Thing_\ '
        NameThing\ 'an"
126 L"d_has_\r\n\t\tadditionally_its_own_Parameters." )]
127    public ref class NameDerivedThing : public AbstractionLevels::
        NameAbstractionLevelRepository::ThingTypes::NameThing
128    {

```

```
129
130     private: Surface^ _nameParameterSurface;
131
132     private: Percent^ _nameParameterPercent;
133
134     public: [System::ComponentModel::Description(L"The_
        benchmark_addes_all_values_from_'Area'\r\n\t\tfrom_
        all_the_'NameDerivedThing\'_to_"
135 L"\r\n\t\tThe_total_'AreaThings\'"
        L"is_measured_in_sqm.")]
136     property Surface^ NameParameterSurface
137     {
138     {
139         Surface^ get();
140         System::Void set(Surface^ value);
141     }
142
143     public: [System::ComponentModel::Description(L"")]
144     property Percent^ NameParameterPercent
145     {
146     {
147         Percent^ get();
148         System::Void set(Percent^ value);
149     }
150 }
151 }
152 }
153 namespace AbstractionLevels {
154     namespace NameAbstractionLevelRepository {
155         namespace ThingTypes {
156
157
158         inline Text^ NameThing::NameParameterText::get()
159         {
160             return this->_nameParameterText;
161         }
162         inline System::Void NameThing::NameParameterText::set(Text^
            value)
163         {
164             this->_nameParameterText = value;
165         }
166
167         inline System::Number^ NameThing::NameParameterNumber::get()
168         {
169             return this->_nameParameterNumber;
170         }
171         inline System::Void NameThing::NameParameterNumber::set(System
            ::Number^ value)
172         {
173             this->_nameParameterNumber = value;
174         }
175     }
```

```

176     inline AbstractionLevels::NameAbstractionLevelRepository::
      ParameterTypes::NameParameterType NameThing::
      NameNameParameterType::get()
177     {
178         return this->_nameNameParameterType;
179     }
180     inline System::Void NameThing::NameNameParameterType::set (
      AbstractionLevels::NameAbstractionLevelRepository::
      ParameterTypes::NameParameterType value)
181     {
182         this->_nameNameParameterType = value;
183     }
184
185
186     inline Surface^ NameDerivedThing::NameParameterSurface::get()
187     {
188         return this->_nameParameterSurface;
189     }
190     inline System::Void NameDerivedThing::NameParameterSurface::
      set (Surface^ value)
191     {
192         this->_nameParameterSurface = value;
193     }
194
195     inline Percent^ NameDerivedThing::NameParameterPercent::get()
196     {
197         return this->_nameParameterPercent;
198     }
199     inline System::Void NameDerivedThing::NameParameterPercent::
      set (Percent^ value)
200     {
201         this->_nameParameterPercent = value;
202     }
203     }
204 }
205 }
206 namespace AbstractionLevels {
207     namespace NameAbstractionLevelRepository {
208         using namespace System;
209         using namespace AbstractionLevels::
      NameAbstractionLevelRepository::ThingTypes;
210         using namespace System;
211         ref class NameAbstractionLevel;
212
213
214         [System::ComponentModel::Description(L"Your_description_for_
      the_'NameAbstractionLevel'\r\nYou_can_add_to_every_
      object_and"
215 L"description\r\nor_leave_it_empty.")]
216         public ref class NameAbstractionLevel
217         {
218

```

```
219     private: System::Collections::Generic::List<
        AbstractionLevels::NameAbstractionLevelRepository::
        ThingTypes::NameThing^ >^ _nameThingCollection;
220
221     private: System::Collections::Generic::List<
        AbstractionLevels::NameAbstractionLevelRepository::
        ThingTypes::NameDerivedThing^ >^
        _nameDerivedThingCollection;
222
223     public: [System::ComponentModel::Description(L"\ 'NameThing
        \ '_is_described_by_Parameters.")]
224     property System::Collections::Generic::List<
        AbstractionLevels::NameAbstractionLevelRepository::
        ThingTypes::NameThing^ >^ NameThingCollection
225     {
226         System::Collections::Generic::List<AbstractionLevels::
        NameAbstractionLevelRepository:: ThingTypes::
        NameThing^ >^ get();
227         System::Void set(System::Collections::Generic::List<
        AbstractionLevels::NameAbstractionLevelRepository::
        ThingTypes::NameThing^ >^ value);
228     }
229
230     public: [System::ComponentModel::Description(L"\ '
        NameDerivedThing\ '_contains_all_Paramters_\r\n\tfrom_
        the_parent_Thing\ '_NameThing\ '_an"
231 L"d_has_\r\n\tadditionally_its_own_Parameters.")]
232     property System::Collections::Generic::List<
        AbstractionLevels::NameAbstractionLevelRepository::
        ThingTypes::NameDerivedThing^ >^
        NameDerivedThingCollection
233     {
234         System::Collections::Generic::List<AbstractionLevels::
        NameAbstractionLevelRepository:: ThingTypes::
        NameDerivedThing^ >^ get();
235         System::Void set(System::Collections::Generic::List<
        AbstractionLevels::NameAbstractionLevelRepository::
        ThingTypes::NameDerivedThing^ >^ value);
236     }
237 };
238 }
239 }
240 namespace AbstractionLevels {
241     namespace NameAbstractionLevelRepository {
242
243
244         inline System::Collections::Generic::List<AbstractionLevels::
        NameAbstractionLevelRepository:: ThingTypes::NameThing^ >^
        NameAbstractionLevel::NameThingCollection::get()
245     {
246         return this->_nameThingCollection;
247     }
```

```
248     inline System::Void NameAbstractionLevel::NameThingCollection
        :: set (System:: Collections:: Generic:: List<AbstractionLevels
        :: NameAbstractionLevelRepository:: ThingTypes:: NameThing^ >^
            value)
249     {
250     }
251
252     inline System:: Collections:: Generic:: List<AbstractionLevels::
        NameAbstractionLevelRepository:: ThingTypes::
        NameDerivedThing^ >^ NameAbstractionLevel::
        NameDerivedThingCollection:: get ()
253     {
254         return this->_nameDerivedThingCollection;
255     }
256     inline System::Void NameAbstractionLevel::
        NameDerivedThingCollection:: set (System:: Collections::
        Generic:: List<AbstractionLevels::
        NameAbstractionLevelRepository:: ThingTypes::
        NameDerivedThing^ >^ value)
257     {
258     }
259     }
260 }
261 namespace AbstractionLevels {
262     namespace NameDerivedAbstractionLevelRepository {
263         namespace ParameterTypes {
264             using namespace System;
265             using namespace System;
266
267         }
268     }
269 }
270 namespace AbstractionLevels {
271     namespace NameDerivedAbstractionLevelRepository {
272         namespace ParameterTypes {
273         }
274     }
275 }
276 namespace AbstractionLevels {
277     namespace NameDerivedAbstractionLevelRepository {
278         namespace ThingTypes {
279             using namespace System;
280             using namespace AbstractionLevels::
                NameDerivedAbstractionLevelRepository:: ParameterTypes;
281             using namespace SemIOLibrary:: Parameters;
282             using namespace SemIOLibrary:: Parameters:: GeometryParameters;
283             using namespace SemIOLibrary:: Parameters:: GeometryParameters::
                Surfaces;
284             using namespace SemIOLibrary:: Parameters:: GeometryParameters::
                Solids;
285             using namespace SemIOLibrary:: Parameters:: GeometryParameters::
                Points;
```

```
286     using namespace SemIOLibrary::Parameters::GeometryParameters::
        Curves;
287     using namespace SemIOLibrary::Parameters::BaseParameters;
288     using namespace SemIOLibrary::Parameters::BaseParameters::
        Numbers;
289     using namespace System;
290     ref class NameThing;
291
292
293     [System::ComponentModel::Description(L"")]
294     public ref class NameThing
295     {
296
297         private: Solid^ _nameParameterSolid;
298
299         public: [System::ComponentModel::Description(L"")]
300         property Solid^ NameParameterSolid
301         {
302             Solid^ get();
303             System::Void set(Solid^ value);
304         }
305     };
306
307     }
308 }
309 namespace AbstractionLevels {
310     namespace NameDerivedAbstractionLevelRepository {
311         namespace ThingTypes {
312
313
314             inline Solid^ NameThing::NameParameterSolid::get()
315             {
316                 return this->_nameParameterSolid;
317             }
318             inline System::Void NameThing::NameParameterSolid::set(Solid^
319                 value)
320             {
321                 this->_nameParameterSolid = value;
322             }
323         }
324     }
325 namespace AbstractionLevels {
326     namespace NameDerivedAbstractionLevelRepository {
327         using namespace System;
328         using namespace AbstractionLevels::
329             NameDerivedAbstractionLevelRepository::ThingTypes;
329         using namespace AbstractionLevels::
330             NameAbstractionLevelRepository;
330         using namespace System;
331         ref class NameDerivedAbstractionLevel;
332
333
```

```

334     [System::ComponentModel::Description(L"This_AbstractionLevel_
        is_derived_from_the\r\nAbstractionLevel_above_with_the_name
        _N"
335     L"ameAbstractionLevel")]
336     public ref class NameDerivedAbstractionLevel : public
        AbstractionLevels::NameAbstractionLevelRepository::
        NameAbstractionLevel
337     {
338
339         private: System::Collections::Generic::List<
            AbstractionLevels::
            NameDerivedAbstractionLevelRepository::ThingTypes::
            NameThing^>^ _nameThingCollection;
340
341         public: [System::ComponentModel::Description(L"")]
342         property System::Collections::Generic::List<
            AbstractionLevels::
            NameDerivedAbstractionLevelRepository::ThingTypes::
            NameThing^>^ NameThingCollection
343         {
344             System::Collections::Generic::List<AbstractionLevels::
                NameDerivedAbstractionLevelRepository::ThingTypes::
                NameThing^>^ get();
345             System::Void set(System::Collections::Generic::List<
                AbstractionLevels::
                NameDerivedAbstractionLevelRepository::ThingTypes::
                NameThing^>^ value);
346         }
347     };
348 }
349 }
350 namespace AbstractionLevels {
351     namespace NameDerivedAbstractionLevelRepository {
352
353
354         inline System::Collections::Generic::List<AbstractionLevels::
            NameDerivedAbstractionLevelRepository::ThingTypes::
            NameThing^>^ NameDerivedAbstractionLevel::
            NameThingCollection::get()
355         {
356             return this->_nameThingCollection;
357         }
358         inline System::Void NameDerivedAbstractionLevel::
            NameThingCollection::set(System::Collections::Generic::List<
            AbstractionLevels::NameDerivedAbstractionLevelRepository::
            ThingTypes::NameThing^>^ value)
359         {
360         }
361     }
362 }

```

List of Figures and code

3.1	Framework diagram	17
3.2	Diagram notation	17
3.3	Project diagram	18
3.4	Abstraction level diagram	18
3.5	Design part diagram	21
3.6	Evaluation part diagram	25
3.7	Interval diagrams	27
3.8	Examples of remapping diagrams	27
4.1	Parts of the framework covered by the prototype	31
4.2	Pipeline prototype	32
A.1	<i>cad-market-revenue-share-worldwide-2016</i>	I
A.2	<i>bim-software-used-by-architectural-companies-japan-2019</i>	II
A.3	<i>global-spatial-information-analytics-tools-software-revenue-by-vendor-2012-2014</i>	III
4.1	SemIO code Syntax demonstration	29
4.2	SemIO code example	34
A.1	Generated C++ code from compiled SemIO code	IV

Bibliography

- Advanced Micro Devices, I. (2020), ‘AMD Athlon Silver 3050U’. Last Access: 2020-10-20.
URL: <https://www.amd.com/de/products/apu/amd-athlon-silver-3050u>
- Alexander, C. (1964), ‘Notes on the synthesis of form’.
- Alexander, C. (2002), *The nature of order: The phenomenon of life*, Taylor & Francis.
- Alexander, C., Ishikawa, S. & Silverstein, M. (1977), ‘A pattern language : towns, buildings, construction’.
- Arup (2012), ‘BIM Maturity measure’. Last Access: 2020-10-20.
URL: <https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/TC1/HTML/ifcproductextension/lexical/ifczone.htm>
- Autodesk (2020a), ‘Autocad for max & windows | 2d/3d cad software | autodesk’. Last Access: 2020-10-20.
URL: <https://www.autodesk.com/products/autocad/overview>
- Autodesk (2020b), ‘Dxf overview’. Last Access: 2020-10-20.
URL: <https://www.autodesk.com/techpubs/autocad/acad2000/dxf/index.htm>
- Autodesk, Inc (2020), ‘Visual scripting environment for designers | Dynamo’. Last Access: 2020-10-20.
URL: <https://dynamobim.org>
- Boehme GbR, D. H. S. (2020), ‘AMD Athlon Silver 3050U - Benchmark und Technische Daten’. Last Access: 2020-10-20.
URL: https://www.cpu-monkey.com/de/cpu-amd_athlon_silver_3050u-1241
- Bojanić, P. (2019), ‘The acts of project(ion) / project acts or projects’, *Rivista di estetica* **71**(71), 92–100.
- buildingSMART International, Ltd (2020a), ‘IfcZone’. Last Access: 2020-10-20.
URL: <https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/TC1/HTML/ifcproductextension/lexical/ifczone.htm>
- buildingSMART International, Ltd (2020b), ‘Industry Foundation Classes (IFC) - buildingSMART Technical’. Last Access: 2020-10-20.
URL: <https://technical.buildingsmart.org/standards/ifc>

- Carmo, M. (2017), *The Second Digital Turn: Design Beyond Intelligence*, The MIT Press.
- Deleuze, G. & Guattari, F. (1994), *What is philosophy?*, Columbia University Press.
- Esri (2020), ‘Advanced 3d city design software | arcgis cityengine’. Last Access: 2020-10-20.
URL: <https://www.esri.com/en-us/arcgis/products/arcgis-cityengine/overview>
- Frampton, D., Blackburn, S., Cheng, P., Garner, R., Grove, D., Moss, J. & Salishev, S. (2009), Demystifying magic: high-level low-level programming, VEE '09, ACM, pp. 81–90.
- Free Software Foundation (2020), ‘The GNU General Public License v3.0 - GNU Project - Free Software Foundation’. Last Access: 2020-10-20.
URL: <https://opensource.org/licenses/mit-license.php>
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994), ‘Design patterns : elements of reusable object oriented software’.
- Garcia, M. (2014), ‘Histories, theories and futures of the details of architecture’, *Architectural design* **84**(4), 14–25.
- German Federal Highway Research Institute (2020), ‘Objekt katalog für das Straßen- und Verkehrswesen’. Last Access: 2020-10-20.
URL: <http://www.okstra.de/>
- German Federal Ministry of the Interior, Building and Community (2015), ‘Guideline for Sustainable Building’. Last Access: 2020-10-20.
URL: https://www.nachhaltigesbauen.de/fileadmin/publikationen/BBSR_LFNB_E_komplett_KOR1_190503.pdf
- German Federal Ministry of Transport and Digital Infrastructure (2015), ‘Road Map for Digital Design and Construction’. Last Access: 2020-10-20.
URL: https://www.bmvi.de/SharedDocs/EN/publications/road-map-for-digital-design-and-construction.pdf?__blob=publicationFile
- Geyer, P., Singh, M. M. & Singaravel, S. (2018), Component-based machine learning for energy performance prediction by multilod models in the early phases of building design, *in* ‘Workshop of the European Group for Intelligent Computing in Engineering’, Springer, pp. 516–534.
- Goodhouse, A. & Canadian Centre for Architecture (2017), ‘When is the digital in architecture?’.
- Goodman, N. (1976), ‘Languages of art : an approach to a theory of symbols’.
- Graphisoft (2020), ‘Archicad - graphisoft’. Last Access: 2020-10-20.
URL: <https://graphisoft.com/solutions/products/archicad>
- Harper, D. (2020a), ‘adequate | Origin and meaning of adequate by Online Etymology Dictionary’. Last Access: 2020-10-20.
URL: <https://www.etymonline.com/word/adequate>

- Harper, D. (2020*b*), ‘bio’. Last Access: 2020-10-20.
URL: <https://www.etymonline.com/columns/post/bio>
- Harper, D. (2020*c*), ‘design| Origin and meaning of designby Online Etymology Dictionary’. Last Access: 2020-10-20.
URL: <https://www.etymonline.com/word/design>
- Harper, D. (2020*d*), ‘model | Origin and meaning of adequate by Online Etymology Dictionary’. Last Access: 2020-10-20.
URL: <https://www.etymonline.com/word/model>
- Harper, D. (2020*e*), ‘project | Origin and meaning of designby Online Etymology Dictionary’. Last Access: 2020-10-20.
URL: <https://www.etymonline.com/word/project>
- Kant, I. (1788*a*), *Kritik der reinen Vernunft*, Felix meiner. unchanged print from edition Rymund Schmidt, published 1956.
- Kant, I. (1788*b*), *THE CRITIQUE OF PURE REASON*. translated by J. M. D. Meiklejohn, published 1885.
- Kurrer, K.-E. (2001), ‘Konrad zuse. der vater des computers (j. alex u. a.)’, *Der Stahlbau* **70**(8), 577–577.
- Latour, B. (2012), A cautious prometheus?: A few steps toward a philosophy of design with special attention to peter sloterdijk, in ‘In Medias Res’, Amsterdam University Press, p. 151.
- Lee, H., Kim, J., Shin, M., Kim, I. & Lee, J.-K. (2014), A demonstration of bim-enabled quantitative circulation analysis using bera language, in ‘ISARC. Proceedings of the International Symposium on Automation and Robotics in Construction’, Vol. 31, IAARC Publications, p. 1.
- Mernik, M., Heering, J. & Sloane, A. (2005), ‘When and how to develop domain-specific languages’, *ACM computing surveys* **37**(4), 316–344.
- Microsoft (2020*a*), ‘C# docs - get started, tutorials, reference. | microsoft docs’. Last Access: 2020-10-20.
URL: <https://dotnet.microsoft.com/learn/dotnet/what-is-dotnet>
- Microsoft (2020*b*), ‘Type float’. Last Access: 2020-10-20.
URL: <https://docs.microsoft.com/en-us/cpp/c-language/type-float>
- Microsoft (2020*c*), ‘What is .net? an open-source developer platform.’. Last Access: 2020-10-20.
URL: <https://dotnet.microsoft.com/learn/dotnet/what-is-dotnet>
- Müller, P., Wonka, P., Haegler, S., Ulmer, A. & Van Gool, L. (2006), Procedural modeling of buildings, in ‘ACM SIGGRAPH 2006 Papers’, pp. 614–623.
- Object Management Group (2020), ‘About the Unified Modeling Language Specification Version 2.5.1’. Last Access: 2020-10-20.
URL: <https://www.omg.org/spec/UML>

- Open Source Initiative (2020), ‘The MIT License | Open Source Initiative’. Last Access: 2020-10-20.
URL: <https://opensource.org/licenses/mit-license.php>
- Parr, T. (2014), ‘ANTLR’. Last Access: 2020-10-20.
URL: <https://www.antlr.org/>
- Pfeifer, W. et al. (2020a), ‘›Design‹ in: Etymologisches Wörterbuch des Deutschen im DWDS’. Last Access: 2020-10-20.
URL: <https://www.dwds.de/wb/etymwb/Design>
- Pfeifer, W. et al. (2020b), ‘›Modell‹ in: Etymologisches Wörterbuch des Deutschen im DWDS’. Last Access: 2020-10-20.
URL: <https://www.dwds.de/wb/etymwb/Modell>
- Pfeifer, W. et al. (2020c), ‘›Projekt‹ in: Etymologisches Wörterbuch des Deutschen im DWDS’. Last Access: 2020-10-20.
URL: <https://www.dwds.de/wb/etymwb/Projekt>
- Python Software Foundation (2020), ‘General python faq – python 3.9.0 documentation’. Last Access: 2020-10-20.
URL: <https://docs.python.org/3/faq/general.html#what-is-python-good-for>
- Robert McNeel & Associates (2020), ‘Grasshopper - New in Rhino 6’. Last Access: 2020-10-20.
URL: <https://www.rhino3d.com/6/new/grasshopper>
- Rojas, R. (1996), ‘Die Architektur der Rechenmaschinen Z1 und Z3 von Konrad Zuse’, *Informatik-Spektrum* **19**(6), 303–315.
- Software Freedom Conservancy (2020), ‘Git’. Last Access: 2020-10-20.
URL: <https://git-scm.com/>
- Spinellis, D. (2001), ‘Notable design patterns for domain-specific languages’, *Journal of systems and software* **56**(1), 91–99.
- Stiny, G. & Gips, J. (1971), Shape grammars and the generative specification of painting and sculpture., in ‘IFIP Congress (2)’, Vol. 2, pp. 125–135.
- Turing, A. M. (1937), ‘On computable numbers, with an application to the entscheidungsproblem’, *Proceedings of the London Mathematical Society* **s2-42**(1), 230–265.
- University Corporation for Atmospheric Research, (2020), ‘NetCDF: The NetCDF Programming APIs’. Last Access: 2020-10-20.
URL: https://www.unidata.ucar.edu/software/netcdf/docs/netcdf_apis.html
- Van Deursen, A. & Klint, P. (2002), ‘Domain-specific language design requires feature descriptions’, *Journal of computing and information technology* **10**(1), 1–17.
- Vesnić, S. (2017), ‘What is an architectural concept? the “concept” of deleuze and “project” of eisenman’, *Filozofija i društvo (Zbornik radova)* **28**(4), 1122–1135.
- Vesnić, S. & Ćipranić, M. (2019), ‘The concept: a map for generations’, *Rivista di estetica* **71**(71), 101–116.

BIBLIOGRAPHY

Zahedi, A. & Petzold, F. (2018), ‘Utilization of simulation tools in early design phases through adaptive detailing strategies’, *Proceedings of the 23rd CAADRIA, Beijing, China* pp. 11–20.